Velox: Scalable Fair Asynchronous MPC from Lightweight Cryptography

Akhil Bandarupalli
Purdue University
West Lafayette, IN, USA
abandaru@purdue.edu

Xiaoyu Ji
Tsinghua University
Beijing, China
jixy23@mails.tsinghua.edu.cn

Aniket Kate
Purdue University & Supra Research
West Lafayette, IN, USA
aniket@purdue.edu

Chen-Da Liu-Zhang
Lucerne University of Applied Sciences and Arts & Web3 Foundation
Lucerne, Switzerland
chen-da.liuzhang@hslu.ch

Daniel Pöllmann
ETH Zurich & Lucerne University of Applied Sciences and Arts
Lucerne, Switzerland
dpoellmann@ethz.ch

Yifan Song
Tsinghua University & Shanghai Qi Zhi Institute
Beijing, China
yfsong@mail.tsinghua.edu.cn

ABSTRACT

Multi-party computation (MPC) enables a set of mutually n distrusting parties to compute any function on their private inputs. Mainly, MPC facilitates agreement on the function's output while preserving the secrecy of honest inputs, even against a subset of t parties controlled by an adversary. With applications spanning from anonymous broadcast to private auctions, MPC is considered a cornerstone of distributed cryptography, and significant research efforts have been aimed at making MPC practical in the last decade. However, most libraries either make strong assumptions like the network being bounded synchronous, or incur high computation overhead from the extensive use of expensive public-key operations that prevent them from scaling beyond a few dozen parties.

This work presents *Velox*, an asynchronous MPC protocol that offers fairness against an optimal adversary corrupting up to $t < \frac{n}{3}$ parties. Velox significantly enhances practicality by leveraging lightweight cryptographic primitives—such as symmetric-key encryption and hash functions—which are 2-3 orders of magnitude faster than public-key operations, resulting in substantial computational efficiency. Moreover, Velox is highly communication-efficient, with linear amortized communication relative to circuit size and only $O(n^3)$ field elements of additive overhead. Concretely, Velox requires just 9.33 field elements per party per multiplication gate, more than 10× reduction compared to the state of the art. Moreover, Velox also offers Post-Quantum Security as lightweight cryptographic primitives retain their security against a quantum adversary.

We implement Velox comprehensively, covering both offline and online phases, and evaluate its performance on a geographically distributed testbed through a real-world application: anonymous broadcast. Our implementation securely shuffles a batch of $k = 256$ messages in 4 seconds with $n = 16$ parties and 18 seconds with $n = 64$ parties, a 36× and 28.6× reduction in latency compared to the prior best work. At scale with $n = 112$ parties, Velox is able to shuffle the same batch of messages in under 50 seconds from end to end, illustrating its effectiveness and scalability. Overall, our work removes significant barriers faced by prior asynchronous MPC solutions, making asynchronous MPC practical and efficient for large-scale deployments involving 100s of parties.

CCS CONCEPTS

• **Security and privacy → Distributed systems security**.

KEYWORDS

Asynchronous Multi-Party Computation (MPC); Asynchronous

1 INTRODUCTION

In the problem of Multi-Party Computation (MPC) [Yao82, GMW87, BGW88, CCD88, RB89], n mutually distrustful parties compute a function on their private inputs, with the guarantee that an adversary does not learn any information about the private inputs apart from what can be deduced from the output.

MPC has been a focus of cryptographic research for over four decades now, with the past ten years bringing significant advances toward practical implementation. Despite this progress, most existing MPC systems still depend on strong communication assumptions—particularly some form of (bounded) synchrony and broadcast channels—that limit their real-world applicability, especially when latency overhead is critical.

In the synchronous communication model, messages are assumed to be delivered within a fixed time frame. However, in real-world high-throughput, low-latency applications, such rigid timing guarantees cannot be ensured. Networks can experience unpredictable delays, which synchronous MPC systems cannot handle effectively, resulting in liveness issues and more severely eliminating non-malicious slow parties.

While asynchronous MPC protocols address such scenarios as they tolerate any delays, current protocols cannot scale well beyond ten of parties. On the one hand, there is a beautiful line of work on information-theoretic protocols [BKR94, PCR10, PCR08, CP23, GLZS24] where substantial progress have been made, but their communication complexity is still impractical. The best-known IT protocol with optimal resilience $t < n/3$ incurs an additive overhead of $O(n^{14})$ [GLZS24]. On the other hand, state of the art practical computational protocols [LYK+19, SLL+24] rely on significant usage of public-key cryptography and/or non-interactive zero-knowledge proofs, and their computational cost constitute their primary efficiency bottleneck.

Recent works [BBB+24, SS24, DDL+24, Mom24, BJK+25] have considered the intermediate setting of *lightweight cryptography*

[SS24]. These protocols only make use of computationally efficient cryptographic primitives such as hash functions and symmetric key encryption, rather than public-key primitives. Their motivation for using such primitives is two-fold: 1) Reducing drastically the computational cost: Each operation with a standard hash function such as `SHA3` or symmetric-key encryption such as `AES` is at least two or three orders of magnitude faster than simple public-key cryptographic operations such as a discrete-log exponentiation; and 2) the overall protocol remains post-quantum secure. Using such tools effectively is nevertheless challenging, since they do not support homomorphic operations and as such it is not easy to obtain protocols with low communication complexity.

Recent attempts at building MPC protocols from lightweight cryptography [Mom24, BJK+25] incur high communication in exchange for lower computational complexity. More concretely, the protocol [Mom24] incurs $O(n^2C + \kappa n^6)$ bits of communication for a circuit of size C, which was recently improved in [BJK+25] to $O(nC + \kappa n^4)$ bits. Although these protocols made a significant step towards making asynchronous MPC practical, their multiplicative constant of more than 100 (even when considering security with abort) and their additive term prevent them from scaling the protocol to the order of a hundred parties.

1.1 Our Contributions

In this work, we investigate whether it is possible to design an asynchronous MPC protocol with optimal resilience $t < n/3$ [BOKR94, ADS20] against active corruptions that scales to larger number of parties, e.g. one hundred parties.

Can we achieve optimally-resilient AMPC that scales to one hundred parties?

We answer in the affirmative by presenting *Velox*, the first AMPC protocol based on lightweight tools (hash functions and symmetric encryption) that scales to one hundred parties. Velox achieves fairness[1] and incurs $O(nC + \kappa n^3)$ bits with practical constants. First, the multiplicative constant is only 9.33 elements per party per multiplication gate. This is a reduction of more than 10× compared to the state of the art. Second, the additive overhead of $O(n^3)$ elements is minimal since any AMPC requires each party to reliably broadcast their input, and the cost for each reliable broadcast incurs inherently a quadratic term $O(n^2)$ [DR85]. See Table 1 for a comparison with state of the art protocols.

THEOREM 1.1. *Let $n = 3t+1$ and κ denote the security parameter. For a finite field $\mathbb{F}$ of size $2^{\Omega(\kappa)}$ and any circuit C of size $|C|$ and depth D, assuming random oracles, there is an AMPC protocol computing the circuit that is secure against at most t corrupted parties with security with fairness. Let C_O be the output size of C, the achieved communication complexity is $O(|C| \cdot n + (D + C_O) \cdot n^2 + n^3)$ field elements, and the expected round complexity is $O(D + \log |C|)$.*

We implemented Velox from end-to-end and benchmarked its performance via the application of anonymous broadcast, where a set of clients broadcast information anonymously. In Figure 1, we show the runtime of the different phases of Velox for $n = 112$ parties and varying sizes of the anonymity set. For an anonymity set size of $k = 256$, our overall runtime is 48.06 seconds. We compare Velox with the preprocessing phase of the prior best asynchronous MPC protocol DumboMPC [SLL+24] (USENIX Security'24) and show that our protocol is faster by a factor of 36 for $n = 16$ and 28.6 for $n = 64$ as shown in Figure 2.

[1]This means that the adversary may abort the protocol, but if it does so, does not learn the output of the computation. Even with security-with-abort, it is folklore knowledge that $t < n/3$ is optimal in the asynchronous setting.

1.2 Related Work

Since seminal works of [BCG93, BKR94] introducing the first asynchronous MPC protocols, significant efforts have been made to improve their efficiency.

Most works focused on improving the asymptotic efficiency, in particular the multiplicative term dependent on the circuit size. There has been a line of work studying protocols achieving information-theoretic security [PCR10, PCR08, CP23, GLZS24, JLS24, AAPP24], where very recently there have been protocols achieving a linear communication term in the circuit size C: $O(nC + n^{14})$ field elements for the case of statistical security and $t < n/3$ corruptions [GLZS24, JLS24], and $O(nC+n^5)$ with perfect security and $t < n/4$ corruptions [AAPP24].

Using computational assumptions, protocols with optimal resilience $t < n/3$ under different tools have been proposed, typically in the form of threshold encryption and non-interactive zero-knowledge proofs [HNP05, HNP08, CHLZ21, Coh16, BKLZL20, CP15], homomorphic commitments [SLL+24], or lightweight cryptography (i.e., hashes and symmetric encryption) [Mom24, BJK+25].

The AMPC protocols [Mom24, BJK+25] require more than $100n$ elements per multiplication gate and also an additive term of at least $O(n^4)$ even for the case of security with abort (without fairness). Our protocol achieving fairness decreases this constant to $9.33n$ elements per multiplication gate and has an additive term of $O(n^3)$.

Finally, the works [LYK+19, DGKN09] introduce AMPC protocols with a preprocessing phase that may not terminate, i.e. they are not live. This undesirable condition is more critical than the standard security with abort, as in the latter case, parties realize that the protocol failed (they obtain $\perp$ as output).

2 TECHNICAL OVERVIEW

In the following, we use n to denote the number of parties and t to denote the corruption threshold. For a non-negative integer $d < n$, we use $[s]_d$ to denote a degree-d Shamir secret sharing of s and $\alpha_0, \ldots, \alpha_n$ to denote distinct field elements. We recap the definition of Shamir secret sharing scheme in Appendix A.4.

2.1 Blueprint for Constructing Asynchronous MPC with Fairness

In this work, we aim to construct a concretely efficient asynchronous MPC with *fairness* in the setting of $n = 3t + 1$, only assuming a random oracle. To be more concrete,

- Our goal is to maintain the linear communication $O(|C| \cdot n)$ as that in [Mom24, BJK+25] while achieving a smaller additive communication overhead of $O(n^3)$.
- On top of that, we want to minimize the constant factor to make it as efficient as the state-of-the-art constructions [DN07, GS20, GLO+21] in the synchronous setting.

Table 1: Comparison of relevant Asynchronous MPC protocols.

Protocol	Post Quantum Security	Computation Complexity	Communication		Security Guarantees	Setup	Cryptographic Assumption
			Amortized Cost Per Multiplication	Additive Overhead			
[SLL+24]‡	✗	$O(n^2\|C\|)$ DLE	$O(n^2)$ ‡	$O(\kappa n^3)$	G.O.D.	PKI + SRS	DLog + q-SDH
[Mom24]	✓	$O(n^2\|C\|)$ LWC	$O(n^2)$	$O(\kappa n^5)$	G.O.D.	Secure Channels	RO
[BJK+25]	✓	$O(n\|C\|)$ LWC	$>100n$	$O(\kappa n^4)$	G.O.D.	Secure Channels	RO
[Mom24]	✓	$O(n\|C\|)$ LWC	$>100n$	$O(\kappa n^4)$	Security with Abort	Secure Channels	RO
Velox	✓	$O(n\|C\|)$ LWC	$\sim 9.33n$	$O(\kappa n^3)$	Fairness	Secure Channels	RO

Computation complexity: The notion $|C|$ denotes the circuit size. The computation cost of our protocols is composed of Lightweight Cryptographic (LWC) operations like symmetric key encryption, cryptographic Hash computations, and pseudorandom number generations. In comparison, each discrete-log exponentiation (DLE) is 100× more expensive than an LWC operation. Each protocol also requires $O(n|C|)$ finite-field arithmetic operations. These operations are cheaper than LWC and DLE operations, so we absorb them into the order notation. **Communication**: The notion κ denotes the security parameter. We compare the number of field elements in the amortized per-gate communication cost and the overall additive overhead. **Security Guarantees.** Fairness guarantees that honest parties and the adversary terminate with the same output (which may be an abort), while Guaranteed Output Delivery (G.O.D.) ensures that the computation never aborts. Security with Abort constructions let the adversary to observe the protocol's output and then abort the protocol.
‡ The work [SLL+24] is only an offline phase protocol. We describe its costs here by pairing its offline phase with HoneyBadgerMPC's [LYK+19] online phase. [SLL+24] achieves linear preprocessing communication cost per multiplication when the network is synchronous and all parties are honest. We compare against this variant in our evaluation section.

Unlike the previous works [Mom24, BJK+25] which first prepare random Beaver triples in the preprocessing phase and then use them to compute multiplication gates in the online phase, we try to directly adapt the state-of-the-art MPC protocols [DN07, GS20, GLO+21] in the synchronous setting to the asynchronous network.

In these protocols, all parties first follow the semi-honest DN protocol [DN07] to compute a degree-t Shamir sharing for each wire value in the circuit. To achieve malicious security, all parties together verify the correctness of multiplication gates, and the output is reconstructed only if the check passes. In particular, the multiplication verification protocol in [GS20] only adds up a sub-linear communication overhead. Thus the achieved communication complexity per gate matches the semi-honest DN protocol. On top of these, [GLO+21] further improves both the communication complexity and the round complexity of the semi-honest DN protocol.

In the following, we mainly discuss adapting the DN protocol in the asynchronous setting and first realizing a secure-with-abort version. Later in section 2.4, we introduce how to upgrade it to achieve fairness. The check of multiplication gates follows previous work in [GS20], and we refer the readers to Appendix E for more details. By combining them, we reduce the communication cost per gate to $9.33n$ field elements, along with overall $O(n^3)$ additive overheads. To the best of our knowledge, the current best works [Mom24, BJK+25] require more than $100n$ field elements per gate, along with $O(n^4)$ additive overhead in the same setting.[2]

Review of the DN Multiplication Protocol. We recap the DN technique as follows. For each addition gate in the circuit with input sharings $([x]_t, [y]_t)$, all parties locally compute output sharing $[z]_t = [x]_t + [y]_t$ relying on the linearity of the Shamir secret sharing scheme. For each multiplication gate, all parties do the following.

(1) All parties prepare a pair of random Shamir sharings $([r]_t, [o]_{2t})$ where $[r]_t$ is a random degree-t Shamir sharing and $[o]_{2t}$ is a random degree-$2t$ Shamir sharing of 0, which is also referred to as *double sharings*[3]. This step can be done in the preprocessing phase.
(2) All parties locally compute $[e]_{2t} := [x]_t \cdot [y]_t + [r]_t + [o]_{2t}$.
(3) All parties reconstruct the secret e, and then locally compute $[z]_t := e - [r]_t$.

The correctness follows from the properties of the Shamir secret sharing scheme. As for security, [GIP+14] has shown that the DN multiplication protocol is secure up to an additive attack, i.e., the adversary can only insert an additive error d such that $z = x \cdot y + d$. Such an attack will be detected in the verification phase, and then all parties can abort the protocol in case the check fails.

Therefore, applying the DN technique in the asynchronous setting is reduced to two problems: (1) preparation of *double sharings*, and (2) public reconstruction of a degree-$2t$ Shamir secret sharing $[e]_{2t}$.

Preparation of Double Sharings. In [DN07], random double sharings can be prepared in two steps: (1) Each party first distributes a random pair of double sharings, (2) and all parties then apply a Vandermonde matrix to extract random double sharings that are not known to any single party. To apply the same idea in the asynchronous setting, it is sufficient to construct a protocol that allows a dealer to distribute a Shamir secret sharing to all parties. The main difficulty here is to ensure that eventually all honest parties should obtain their shares from the dealer. Note that directly letting the dealer send the shares does not work since a malicious dealer may only send shares to a subset of honest parties while the rest of honest parties may never receive their shares.

Previous works [Mom24, BJK+25] have proposed solutions for distributing Shamir sharings, while both of them require $O(n^3)$ additive communication overhead. This results in $O(n^4)$ additive communication overhead for the whole AMPC because each party needs to lead one instance. To achieve our goal, we give an overview in section 2.2 of our construction that both improves the additive

[2]Previous works [Mom24, BJK+25] achieve guaranteed output delivery. For a fair comparison, we compare with the case of secure-with-abort construction in their works.

[3]The *double sharing* defined in [DN07] is in the form of $([r]_t, [r]_{2t})$ with the same random secret r. We follow [GLO+21] and consider an equivalent form where the second sharing is replaced by a random degree-$2t$ Shamir sharing of 0.

communication overhead to $O(n^2)$ per instance and reduces the constant multiplicative factor 6 to around 2 for concrete efficiency.

Public Reconstruction of Secret. At a first glance, reconstructing a degree-$2t$ Shamir sharing can be done by letting parties exchange their shares. Since there are at least $2t+1$ honest parties, each party will eventually receive at least $2t+1$ shares, which are sufficient to reconstruct the secret. One subtlety is that corrupted parties may send incorrect shares to honest parties. As a result honest parties may not receive the correct reconstruction result. In the worst case, different honest parties may end up with different reconstruction results. In the synchronous setting, since a degree-$2t$ Shamir sharing is fully determined by honest parties' shares, any incorrect share sent by a corrupted party can be detected with all parties' shares. However in the asynchronous setting, each party needs to proceed after receiving $2t+1$ shares to achieve liveness, which means that honest parties cannot hope to receive all parties' shares to detect incorrect shares.

In Appendix B.1, we show that the above attempt is insecure when evaluating two layers of multiplications in the asynchronous setting by giving a concrete attack. We note that the security issue comes from honest parties not using the same public reconstruction result. That is, even if the reconstruction result is incorrect, as long as all honest parties use the same reconstruction result, the protocol remains secure before reconstructing the output, which allows us to postpone the check of the reconstructions to the end of the protocol and make use of the multiplication verification protocol [GS20] to achieve malicious security efficiently.

To achieve the same reconstruction results among all honest parties, previous works [Mom24, BJK+25] propose two different solutions:

- In [Mom24], all parties use an ACS protocol to agree on the same $2t+1$ shares of $[e]_{2t}$ to recover e.
- In [BJK+25], $[e]_{2t}$ is first reconstructed to a special party P_{king}. Then all parties use the reconstruction result broadcast by P_{king}.

Both of their solutions guarantee that all honest parties reconstruct the same result, but incur an additive communication overhead of $O(n^3)$. Note that both [Mom24, BJK+25] use the above approach when preparing random Beaver triples, which can be viewed as a depth-1 circuit. In our case, if we use the solutions in [Mom24, BJK+25], it would incur $O(D \cdot n^3)$ communication where D is the circuit depth.

In section 2.3, we show how to reduce the additive communication overhead of public reconstruction to $O(n^2)$ per layer, thus achieving $O(Dn^2)$ across all layers.

2.2 Generating Random Double Sharings

To prepare random double sharings, previous works [Mom24, BJK+25] make use of a primitive `PrivSend`, which allows all parties to verify the delivery of messages between a sender and a receiver. Then it is sufficient to let the dealer distribute shares via `PrivSend`. In this way, if the sharing phase succeeds, all honest parties will eventually receive their shares from the dealer. Assuming a random oracle, `PrivSend` can be constructed with $O(|m| + n^2)$ communication to send a message of length $|m|$.

However, this approach incurs an additive communication overhead of $O(n^3)$ since the dealer needs to invoke one instance for each receiver. In our case, we can only afford $O(n^2)$ communication overhead for distributing random Shamir sharings. We note that [SS24] has explored a parallel version of `PrivSend` for n distinct receivers when the dealer is the same, and it reduces the additive overhead from $O(n^3)$ to $O(n^2 \log n)$, but there is still an $O(\log n)$ gap. To further reduce the additive overhead, we explore a weaker version of `PrivSend`, which is sufficient for our construction and only costs $O(n^2)$ additive overhead among n instances.

Private Send with Publicly Verifiable Delivery. We denote (A, B) as a pair of sender and receiver, and m be the message sent from A to B. In previous works [Mom24, BJK+25], the idea is to use *asynchronous verifiable information dispersal* (AVID) to deliver a message. The primitive AVID contains two phases: the disperse phase and the retrieve phase. In the disperse phase, the dealer will commit a message. If all parties terminate the disperse phase, then they can later open the message to a receiver in the retrieve phase. AVID is very similar to the requirements for `PrivSend`, we can let A send m to B through AVID, and when all parties terminate the disperse phase, they can acknowledge that A has sent m to B because all parties can recover it to B. The only difference is that AVID does not guarantee the privacy of the messages. To achieve `PrivSend`, we assume that A and B will agree on a common randomness r and A will deliver ciphertext $c := m \oplus r$ to B through AVID. To efficiently prepare r, the idea in [Mom24, BJK+25] is to prepare a random seed between each pair for (A, B), then they apply pseudo-random number generator (PRG) on it to generate r. However, the preparation of seeds for all parties requires $O(n^3)$ additive overhead in [BJK+25]. We will show how to optimize it to $O(n^2)$ in section 4.

For the construction of AVID, the idea is that A divides the message c into $t+1$ parts $c_0, \ldots, c_t$, and encodes them into a degree-t polynomial $f(X)$ as follows:

$$f(X) = c_0 + c_1 \cdot X + \cdots + c_t \cdot X^t.$$

Then A will send an evaluation point $f(\alpha_i)$ to each party P_i, and all parties will execute a reliable agreement (RA) protocol with input 1 if they receive their evaluation points from A. When RA terminates with 1, all parties can guarantee that B will receive the message because at least $t+1$ honest parties have received their evaluation points from A, and can forward them to B to help him reconstruct the message c. Upon getting the ciphertext c, the receiver B can decode $m = c \oplus r$.

The above construction guarantees the delivery of a message, but not correctness. This is because B may receive incorrect points from corrupted parties. To prevent this, previous works let A broadcast the commitment of each party's points. This will cause $O(n^2)$ additive overhead and we can not afford it. To further reduce it to amortized $O(n)$, our idea is that we will use a secure-with-abort version of AVID, which means that B can abort the protocol if some parties deviate from the protocol. Therefore, we will let A only broadcast the commitment of the message he sent to B, and B will abort if the reconstructed result does not match A's commitment. Since A can broadcast the commitment of messages for distinct B in parallel, then the amortized cost per instance is $O(n)$.

For the constant factor depending on $|m|$, the above construction is $2 \cdot \frac{n}{t+1} \approx 6$. In section 4, we show how to further reduce it to 5.

Asynchronous Secret Sharing Scheme. Based on the primitive PrivSend, we can construct the secret sharing scheme by letting the dealer invoke PrivSend for each party to deliver the shares. For degree-$2t$ sharing, this construction is sufficient, and we will prove the security later. For degree-t sharing, all parties need to additionally check whether their shares are consistent and lie on degree-t polynomials.

To address this issue, we follow the idea in [BJK+25] and use the primitive called distributed zero-knowledge proof (DZK). This primitive allows a prover to convince a set of verifiers that their shares lie on a degree-t polynomial. Previous work [ABCP23] realizes DZK based on the assumption of the random oracle, and we refer the reader to Appendix A.3 for the functionality and more details.

Based on DZK and our weaker version of PrivSend, we realize the secret sharing scheme with amortized linear cost and $O(n^2)$ additive overhead, which save a factor of $O(n)$ compared to the previous works [Mom24, BJK+25]. Our next goal is to reduce the constant multiplicative factor, and the idea is to reduce the number of PrivSend during the sharing phase:

(1) The dealer first directly sends shares to each party, and all parties will broadcast a confirmation message when they receive and accept their shares from the dealer.
(2) When the dealer first receives $n-t$ confirmation messages, he broadcasts the set of these parties, and only invokes PrivSend for parties not in this set.
(3) When all parties receive the set from the dealer, they continue after receiving the corresponding confirmation messages from parties in this set. When they terminate PrivSend for parties not in this set, they terminate the sharing phase.

Based on this optimization, we reduce the constant factor from 5 to $1 + 5/3 = 8/3$ at the cost of several additional rounds. For simplicity, here we skip the optimization of how all parties efficiently broadcast their confirmation message. Recall that the reliable broadcast requires $O(n^2)$ communication, then each party can only broadcast once (for all instances of PrivSend led by all dealers) to bound the additive overhead within $O(n^3)$. In section 5, we will show how this is achieved.

The above construction realizes a security-with-abort secret sharing protocol and is sufficient for a secure-with-abort AMPC. Later in section 5, we will show that the degree-t secret sharing scheme is also a *verifiable* secret sharing scheme, which guarantees that when all parties terminate the sharing phase, they can reconstruct the secrets distributed by the dealers. This will be helpful for upgrading a secure-with-abort AMPC to a fairness AMPC, as we will introduce later in section 2.4.

Using PRG to Reduce Communication Complexity. Recall that during PrivSend, each pair of parties will agree on a seed to generate randomness. Then we can also use the randomness to let a fraction of parties generate their shares locally without interaction, and the dealer can compute the same set of shares using the common randomness as well.

- For a random degree-t Shamir sharing, each of the first t parties can locally generate his shares by applying PRG on the common seed with the dealer.
- For a random degree-$2t$ Shamir sharing, similarly, each of the first $2t$ parties can locally generate their shares by applying PRG on the common seed with the dealer. Then the dealer no longer needs to send shares to the last t parties through P2P channels, but directly invoke PrivSend for them.

This optimization helps to reduce the constant factor to $7/3$ for distributing degree-t Shamir sharings and $5/3$ for distributing degree-$2t$ Shamir sharings.

Generating Random Sharings in Pipeline. To ensure liveness in generating random double sharings, all parties first agree on a subset of $n-t$ successful dealers. This causes the double sharings from unselected parties (which may up to t parties) to be wasted. To reduce this overhead, we may generate random sharings in pipeline.

Assuming that all parties need to prepare N random *double sharing*, they divide the generation into k segments. In the first segment, all parties still follow the previous idea to generate N/k random *double sharing*. While in the following $k-1$ segments, only parties whose random double sharings have been used will redistribute new random double sharings. Then at the end of the generation phase, only random double sharings distributed by t parties for a single segment will be wasted. In this way, we reduce the amortized cost of preparing each pair of random double sharings from $3n$ field elements to $(2 + 1/k)n$ field elements.

We refer the readers to Appendix D.5 for more details about this construction. One thing to note is that this is a trade-off between communication complexity and round complexity, because all parties need to execute these k segments in sequence, and this construction increases the round complexity by a factor of k.

2.3 Public Reconstruction and Efficient DN Multiplication Protocol

We start from the solution in [Mom24], for the reconstruction of $2t+1$ multiplication gates $\{([a_i]_t, [b_i]_t\}_{i=1}^{2t+1}$, we prepare $2t+1$ uniformly random double sharing $\{([r_i]_t, [o_i]_{2t}\}_{i=1}^{2t+1}$. Then for $i \in [2t+1]$, we define $[z_i]_{2t} := [a_i]_t \cdot [b_i]_t + [r_i]_t + [o_i]_{2t}$ and a degree-$2t$ polynomial $f(X)$ as follows:

$$f(X) = z_1 + z_2 \cdot X + \cdots + z_{2t+1} \cdot X^{2t}.$$

Then each party will compute his shares of $[f(\alpha_i)]_{2t}$ and send it to P_i for all $i \in [n]$. Each party P_i will use the first $2t+1$ shares of $[f(\alpha_i)]_{2t}$ he received to reconstruct $f(\alpha_i)$. To guarantee that all parties recover the same $f(X)$, the idea in [Mom24] is that all parties will agree on the same $2t+1$ evaluation points $f(\alpha_i)$ for reconstruction relying on an ACS protocol, which costs $O(n^3)$ communication.

We observe that under security with abort, to ensure that all honest parties end up with the same reconstruction results, it is sufficient to let every pair of parties exchange the hash of their reconstruction results. A party will only accept the reconstruction results if he receives at least $2t+1$ hashes that match his reconstruction results. Since any two honest parties will receive hashes from at least $t+1$ common parties, which include at least one honest party, their reconstruction results will be the same with overwhelming probability. Therefore, we can let all parties first exchange their evaluation points $f(\alpha_i)$ and then the hash of their reconstruction results in two rounds.

This solution completes in 3 rounds, with $2n \cdot \frac{n}{2t+1} \approx 3n$ elements of communication per reconstruction in the online phase. In Section 7.1, we further reduce the communication complexity by reducing the number of required *double sharings* using the *t-wise*

independence technique from [GLO+21], which reduces the number of degree-$2t$ Shamir sharings of 0 from $2t+1$ to t.

Based on these optimizations, computing $2t+1$ multiplication gates involves preparing $2t+1$ degree-t random sharings, t degree-$2t$ random sharings of 0, and performing public reconstruction. The amortized communication per multiplication gate is

$$\left(\frac{7}{3}\times 2\cdot(2t+1)+\frac{5}{3}\times 2\cdot t\right)/(2t+1)\cdot n+3n\approx 9.33n$$

field elements, plus an overall additive overhead of $O(n^3)$.

2.4 Upgrading Security-with-Abort to Fairness in a Black-Box Way

To ensure fairness in computing a common output y, we first let all parties run a secure-with-abort AMPC protocol to compute a degree-t Shamir sharing $[y]_t$. If all parties have shares of $[y]_t$, then they can directly exchange their shares and use online error correction (OEC) to guarantee that all parties can recover y. Since not all honest parties are guaranteed to receive valid shares now, directly reconstructing y may leak it only to the adversary. To prevent this, a random mask $[r]_t$ is added to $[y]_t$, and parties publicly reconstruct $y+r$ instead. If all parties agree on $y+r$, they reconstruct r and recover y; otherwise, they abort. Therefore, we reduce this problem to the following tasks. Refer to section 7 for detailed construction.

- **Mask Generation:** All parties generate a random sharing $[r]_t$ that guarantees the reconstruction of r when they need. This is achieved with the help of verifiable secret sharing.
- **Agreement on $y+r$:** Parties exchange shares of $[y+r]_t$ and use OEC to reconstruct. If any party sees a failure, they input $\perp$ to a non-intrusion BA protocol.[4] If the consensus is reached on $y+r$, they reconstruct r and output y; otherwise, they abort.

Other Optimization. We follow the technique in [GLO+21] to reduce the round complexity in the online phase by a factor of 2, thus achieving 1.5 rounds per layer of multiplications. Roughly speaking, the idea is that all parties will evaluate two neighboring layers in parallel. See Appendix B.2 for more details.

3 PRELIMINARIES

We denote the security parameter by κ and require the field size to be $2^{\Omega(\kappa)}$.

3.1 Model

We consider protocols among a set $\mathcal{P}$ of n parties $P_1,\ldots,P_n$. Our protocols are proven secure in the model by Canetti [Can00]. Parties have access to a network of point-to-point asynchronous and secure channels (for details of the asynchronous network model, we refer the reader to [CR98]). Asynchronous channels guarantee *eventual* delivery, meaning that messages sent are eventually delivered, and the adversary does the scheduling of the messages. In particular, the adversary can arbitrarily (but finitely) delay all messages sent and deliver them out of order. We also consider the fully malicious adversary, that can completely control the behavior of corrupted parties.

[4] A non-intrusion BA protocol guarantees that the agreement result is $\perp$ or an honest party' input.

Functionality of Asynchronous MPC. We define the functionality for AMPC with fairness as below.

Functionality $\mathcal{F}_{\text{AMPC-Fair}}$

$\mathcal{F}_{\text{AMPC-Fair}}$ proceeds as follows, running with parties $\mathcal{P}=\{P_1,\ldots,P_n\}$, an adversary $\mathcal{S}$ and a n-party function $f:(\{0,1\}^*\cup\{\perp\})^n\to\{0,1\}^*\cup\{\perp\}$. For each party P_i, initialize an input value $x^{(i)}=\perp$.

1: Upon receiving an input v from $P_i\in\mathcal{P}$, if CoreSet has not been recorded yet or if $P_i\in$ CoreSet, set $x^{(i)}=v$.
2: Upon receiving an input CoreSet from $\mathcal{S}$, verify that CoreSet is a subset of $\mathcal{P}$ of size at least $n-t$, else ignore the message. If CoreSet has not been recorded yet, then record CoreSet and for every $P_i\notin$ CoreSet, set $x^{(i)}=0$.
3: If the CoreSet has been recorded and the value $x^{(i)}$ has been set to a value different from $\perp$ for every $P_i\in$ CoreSet, compute $y=f(x^{(1)},\ldots,x^{(n)})$. Then for each $P_i\in\mathcal{P}$, send a request-based delayed output y to P_i.
- Upon receiving the request `Fail`, if the output has not been delivered to any party, change the output of all parties to `Fail`. Otherwise, ignore this request.

3.2 Agreement Primitives

Our construction makes use of the following agreement primitives and we give the definitions of them in Appendix A.1. Previous works give an efficient construction in the same setting as ours and only assume the random oracle, we will use them to build our protocols in a black-box way.

- **Reliable Broadcast**. Functionality $\mathcal{F}_{\text{rbc}}$ allows the parties to agree on the value of a sender without requiring termination if the sender is corrupted. The authors in [DXR21] give a construction with communication complexity $O(L\cdot n+\kappa\cdot n^2)$ for broadcasting L-bit messages.
- **Byzantine Agreement**. Functionality $\mathcal{F}_{\text{ba}}$ allows parties to agree on a common message. The authors in [MMR15] give a construction with communication complexity $O(n^3)$ for broadcasting 1-bit agreement.
- **Reliable Agreement.** Functionality $\mathcal{F}_{\text{ra}}$ is the agreement version of the reliable broadcast where all parties have input. It guarantees that when all parties terminate, at least $t+1$ honest parties have the same inputs[5]. The authors in [DDL+24] give a construction with communication complexity $O(L\cdot n^2)$ for L-bit agreement.
- **Agreement on a Common Set.** Functionality $\mathcal{F}_{\text{acs}}$ allows parties to agree on a set of at least $n-t$ parties that satisfy a certain property. The authors in [DDL+24] give a construction with communication complexity $O(\kappa\cdot n^3)$.

4 PRIVATE SEND WITH PUBLICLY VERIFIABLE DELIVERY

In this section, we give the construction of `PrivSend` mentioned in section 2.2, which will be used to build our secret sharing scheme in section 5. The achieved expected round complexity is constant, and

[5] The drawback is it only guarantees termination if all honest parties have the *same* inputs.

the communication cost is $O(|m|+n^2)$ with the $|m|$ term specifically having a multiplicative constant of 5.

4.1 Functionality of security with abort Private Send

The functionality of PrivSend is defined below and contains an initialization phase and a dispersal phase. During the initialization phase, it will forward a symmetric key (the random seed mentioned in section 2.2) received from the dealer to the receiver, which will be used during the secret sharing scheme to reduce communication complexity. The dispersal phase can be executed multiple times, each time it forwards the message received from the dealer to the receiver and a confirmation message of delivery to all parties. If the functionality receives an instruction abort from the idea adversary $\mathcal{S}$, then he will change the receiver's output by abort.

Functionality $\mathcal{F}_{\text{PrivSend-Ab}}$

$\mathcal{F}_{\text{PrivSend-Ab}}$ proceeds as follows, running with parties $\mathcal{P} = \{P_1, \ldots, P_n\}$, a dealer D, a receiver R and an adversary $\mathcal{S}$.

1: Upon receiving a message $(\texttt{Init}, \text{key})$ from D, send a request-based delayed message $(\texttt{Init}, \text{key})$ to R and a request-based delayed message $(\texttt{Delivered}, \texttt{Init})$ to all parties.
   - Upon receiving abort from $\mathcal{S}$, if the output of R has not been delivered, change the output of R to $(\texttt{Init}, \texttt{abort})$. Otherwise, ignore this request.
2: Upon receiving a message ID Id from $2t+1$ parties and a message $(m, \texttt{Id})$ from D, send a request-based delayed output $(m, \texttt{Id})$ to R and a request-based delayed message $(\texttt{Delivered}, \texttt{Id})$ to all parties.
   - Upon receiving abort from $\mathcal{S}$, if the output of R has not been delivered, change the output of R to abort. Otherwise, ignore this request.

4.2 Preparation of Random Symmetric Key

For the preparation of the random symmetric key, we let the dealer encode it into a degree-t polynomial and distribute evaluation points to all parties. Then all parties invoke an instance of RA to determine whether a sufficient number of parties have received the evaluation points. When all parties terminate RA with 1, at least $t+1$ honest parties have received the evaluation points and can guarantee to help the receiver recover the symmetric key.

An optimization here is that the dealer can distribute the symmetric key for distinct receivers in parallel, and all parties will invoke only one instance of RA for distinct receivers in parallel. This helps to reduce the amortized cost to $O(n)$ for each receiver.

Protocol Π_{PrepKey}

Let $\alpha_0, \ldots, \alpha_n$ be distinct field elements. The dealer D does the following to distribute a symmetric key to a receiver R.

Distributing Phase

1: The dealer D randomly samples two elements in $\mathbb{F}$ as key, r, then he randomly samples two degree-t polynomials $f(x), g(x)$ such that $f(\alpha_0) = \text{key}, g(\alpha_0) = r$.
2: D sends $f(\alpha_j), g(\alpha_j)$ to each party $P_j \in \mathcal{P}$ and reliably broadcasts $c := H(\text{key}, r)$.
3: All parties jointly invoke an instance of $\mathcal{F}_{\text{ra}}$, each party P_j sets his input as 1 if he receives $f(\alpha_j), g(\alpha_j)$ and c from D. When all parties terminate $\mathcal{F}_{\text{ra}}$ with 1, they move to the Reconstruction Phase.

Reconstruction Phase

4: Each party P_j sends $f(\alpha_j), g(\alpha_j)$ to R.
5: When R receives $f(\alpha_j), g(\alpha_j)$ from $t+1$ distinct P_j, he reconstructs key and r and checks whether $c = H(\text{key}, r)$. If true, he outputs key and abort otherwise.

4.3 Asynchronous Verifiable Information Dispersal

In section 2.2 we have shown the high-level idea of the construction of AVID, and we continue to introduce how to reduce the constant factor from 6 to 5. The idea is that we let each party send a "signature" to the dealer if he received his evaluation points, and the dealer initializes a set to include the parties who provide valid "signatures" to him. When the set is of size $2t+1$, the dealer will broadcast this set to prove that there are $2t+1$ parties claiming that they have received their shares, and all parties can terminate the distributing phase because among these $2t+1$ parties, at least $t+1$ of them are honest and will help the receiver to recover the messages. We reduce the constant factor from 6 to 5 because parties not in this set no longer need to forward their evaluation points to the receiver.

To realize the "signature" under the random oracle assumption, the solution is to let each party locally random sample a value r and then broadcast $H(r)$. When he needs to send a "signature" to the dealer D, he can send the opening r to D, and all parties can verify if the opening provided by D is a correct opening of $H(r)$ broadcast by this party.

Protocol $\Pi_{\text{AVID-Ab}}$

The dealer D takes a message m as input and does the following to send it to R.

Distributing Phase

1: Each party P_i randomly samples r_i and reliably broadcasts $h_i := H(r_i)$.
2: D parses m into $t+1$ parts, each of size $|m|/(t+1)$, denoted by $s_0, \ldots, s_t$. Then D computes $f(x) = s_0 + s_1 \cdot x + \cdots + s_t \cdot x^t$ and sends $f(\alpha_i)$ to party P_i. D also randomly samples a field element r and a degree-t polynomial $g(x)$ such that $g(\alpha_0) = r$.
3: D sends $g(\alpha_i)$ to party P_i and reliably broadcasts $c := H(m, r)$.
4: Each party P_i who receives $f(\alpha_i), g(\alpha_i)$ and c from D will send r_i to D.
5: D initializes a set $\mathcal{W} = \emptyset$, upon receiving h_i and correct opening r_i such that $h_i = H(r_i)$ from P_i, D adds (P_i, r_i) to $\mathcal{W}$. When $|\mathcal{W}| = 2t+1$, D reliably broadcasts $\mathcal{W}$.
6: Upon receiving a valid set $\mathcal{W}$ from D, all parties terminate the distributing phase.

Reconstruction Phase

7: Each party $P_i \in \mathcal{W}$ sends $f(\alpha_i), g(\alpha_i)$ to the receiver R and they terminate the protocol.
8: For the receiver R, when he receives $f(\alpha_i), g(\alpha_i)$ from $t+1$ distinct $P_i \in \mathcal{W}$, he reconstructs $f(x)$ and $g(x)$ and checks whether $c = H(m, r)$. If true, he outputs m and abort otherwise.

To control the additive overhead for the preparation of "signatures", we do the following: (1) each party P_i will execute Step 1 for distinct dealers in parallel to amortize the cost to $O(n)$ for broadcasting each r_i. (2) r_i will be used for distinct R when the dealer D is the same party, and P_i will send r_i to D in Step 3 after he receives $f(\alpha_i)$ from D for all distinct R.

4.4 Construction of PrivSend

By combining the sub-protocols $\Pi_{\texttt{PrepKey}}$ and $\Pi_{\texttt{AVID-Ab}}$, we give the construction of $\Pi_{\texttt{PrivSend-Ab}}$ as follows. Refer to Appendix C for the proof of Lemma 4.1 and detailed cost analysis.

Protocol $\Pi_{\texttt{PrivSend-Ab}}$

The initialization phase will only be executed once when all parties first participate in an instance of $\Pi_{\texttt{PrivSend-Ab}}$ for the sender D and receiver R.

Initialization Phase

1: D leads an instance of $\Pi_{\texttt{PrepKey}}$ and distribute key to R, and R will terminate with key or `abort` at the end of $\Pi_{\texttt{PrepKey}}$.

The disperse phase can be executed multiple times. All parties agree on a unique message id (denoted by `Id`) each time they participate.

Disperse Phase(`Id`)

1: The sender takes a message $m_{\texttt{Id}}$ as input, then he computes $c_{\texttt{Id}} = m_{\texttt{Id}} \oplus H(\text{key}, \texttt{Id})$ and leads an instance of $\Pi_{\texttt{AVID-Ab}}$ with input $c_{\texttt{Id}}$.
2: All parties terminate (with a flag `Delivered`) when they terminate $\Pi_{\texttt{AVID-Ab}}$, R will receive $c_{\texttt{Id}}$ or `abort` at the end of $\Pi_{\texttt{AVID-Ab}}$ and terminates with $m_{\texttt{Id}} = c_{\texttt{Id}} \oplus H(\text{key}, \texttt{Id})$ or `abort`.

LEMMA 4.1. *Protocol* $\Pi_{\texttt{PrivSend-Ab}}$ *securely computes* $\mathcal{F}_{\texttt{PrivSend-Ab}}$ *against a fully malicious adversary* $\mathcal{A}$ *who corrupts at most* $t < n/3$ *parties.*

5 PREPARATION OF SHAMIR SECRET SHARINGS

In this section, we give our construction of the secret sharing schemes. They achieve an amortized linear cost per secret, along with $O(n^2)$ additive overhead.

5.1 Functionality of ACSS with abort and Sh2t

$\mathcal{F}_{\texttt{ACSS-Ab}}, \mathcal{F}_{\texttt{Sh2t-Ab}}$ corresponds to the functionalities of degree-t and $2t$ secret sharing schemes, respectively. For $\mathcal{F}_{\texttt{ACSS-Ab}}$, it supports public reconstruction of dealer's secrets when all parties successfully terminate the sharing phase. Here we consider a party successfully terminates the sharing phase if he receives shares or `abort` from the functionality. If anyone receives a failure symbol `Fail` instead, all parties will also receive the same `Fail`, and they can terminate the whole protocol with fairness.

Functionality $\mathcal{F}_{\texttt{ACSS-Ab}}$

Public Input: $(\alpha_0, \ldots, \alpha_n), L$
$\mathcal{F}_{\texttt{ACSS-Ab}}$ runs with parties $\mathcal{P} = \{P_1, \ldots, P_n\}$, a dealer $D \in \mathcal{P}$, and an adversary $\mathcal{S}$.

1: For honest dealer D, upon receiving secrets $s_1, \ldots, s_L$ from D, the functionality randomly samples degree-t polynomial $q_\ell(x)$ such that $q_\ell(\alpha_0) = s_\ell$ for all $\ell \in [L]$. For corrupted dealer D, the functionality waits to receive L degree-t polynomials $q_1(\cdot), \ldots, q_L(\cdot)$ from D.
2: For each $P_i \in \mathcal{P}$, the functionality sends a request-based delayed output $q_1(\alpha_i), \ldots, q_L(\alpha_i)$ to P_i.
   - Upon receiving a request $(\texttt{abort}, P_i)$ from $\mathcal{S}$, if the output of P_i has not been delivered, change the output of P_i to `abort`. Otherwise, ignore this request.
   - Upon receiving a request `Fail` from $\mathcal{S}$, if the output of all parties has not been delivered, change the output of all parties to `Fail`. Otherwise, ignore this request.
3: Upon receiving `Public-Recon` from $t+1$ parties, if it has not received `Fail` from $\mathcal{S}$ before all parties' outputs are delivered, it sends a request-based delayed output $q_1(\cdot), \ldots, q_L(\cdot)$ to all parties.

Functionality $\mathcal{F}_{\texttt{Sh2t-Ab}}$

Public Input: $(\alpha_0, \ldots, \alpha_n), L$
$\mathcal{F}_{\texttt{Sh2t-Ab}}$ runs with parties $\mathcal{P} = \{P_1, \ldots, P_n\}$, a dealer $D \in \mathcal{P}$, and an adversary $\mathcal{S}$.

1: For honest dealer D, upon receiving secrets $s_1, \ldots, s_L$ from D, the functionality randomly samples degree-$2t$ polynomial $q_\ell(x)$ such that $q_\ell(\alpha_0) = s_\ell$ for all $\ell \in [L]$. For corrupted dealer D, the functionality waits to receive L polynomials $q_1(\cdot), \ldots, q_L(\cdot)$ from D.
2: For each $P_i \in \mathcal{P}$, the functionality sends a request-based delayed output $q_1(\alpha_i), \ldots, q_L(\alpha_i)$ to P_i.
   - Upon receiving a request $(\texttt{abort}, P_i)$ from $\mathcal{S}$, if the output of P_i has not been delivered, change the output of P_i to `abort`. Otherwise, ignore this request.

5.2 Construction of ACSS with abort and Sh2t

The following protocols $\Pi_{\texttt{ACSS-Ab}}, \Pi_{\texttt{Sh2t-Ab}}$ realize $\mathcal{F}_{\texttt{ACSS-Ab}}, \mathcal{F}_{\texttt{Sh2t-Ab}}$, respectively. For $\Pi_{\texttt{ACSS-Ab}}$, it will also be a verifiable secret sharing scheme because if all parties terminate the sharing phase (including distributing, verification and termination phase), then they will receive a valid set $\mathcal{M}$, which includes at least $t+1$ honest parties who have accepted their shares. Later in the the public reconstruction phase, these $t+1$ honest parties can verifiably send their shares to all parties with the help of DZK, then all parties can eventually receive these $t+1$ parties' shares can recover the secrets.

One thing to note is that if we use PRG to optimize the communication, the first t parties may be unable to generate their shares from the random seed, since $\mathcal{F}_{\texttt{PrivSend-Ab}}$ only achieves security with abort. Then we cannot guarantee the termination for an honest dealer, since the honest parties among the first t parties will not get their shares. To address this issue, we will let these parties send a failure symbol to the dealer, and the dealer will reliably broadcast it to let all parties terminate the sharing phase. To distinguish from the case parties terminate with `abort` but the sharing phase is still successful, we will use `Fail` to denote the failure symbol.

Protocol $\Pi_{\mathsf{ACSS\text{-}Ab}}$

Let $\alpha_0, \ldots, \alpha_n$ be distinct field elements, L be the number of secrets. Denote $\mathcal{F}^{(i)}_{\mathsf{PrivSend\text{-}Ab}}$ as the instance of $\mathcal{F}_{\mathsf{PrivSend\text{-}Ab}}$ and between the dealer D and each P_i. Let key_i be the symmetric key sent from D to P_i during $\mathcal{F}^{(i)}_{\mathsf{PrivSend\text{-}Ab}}$.

Distributing Phase

1: D possesses a list of L secrets $s_1, \ldots, s_L$ as inputs. For $i \in [t]$, D expands $H(\mathsf{key}_i)$ to get L random values, denoted by $\{h^{(i)}_\ell\}^L_{\ell=1}$. For each secret s_ℓ, D computes degree-t polynomial $f_\ell(x)$ such that $f_\ell(\alpha_0) = s_\ell$ and $f_\ell(\alpha_i) = h^{(i)}_\ell$ for $i \in [t]$.

2: Let $m_i := \langle Shares, f_1(\alpha_i), \ldots, f_L(\alpha_i)\rangle$, for $i \in [t+1, n]$, D sends m_i to P_i.

3: Invokes an instance of $\mathcal{F}_{\mathsf{dZK}}$ with inputs $f_1(x), \ldots, f_L(x)$.

Verification Phase

1: Each party P_i randomly samples a value r_i and reliably broadcast $H(r_i)$. This step can be executed in parallel for different dealers.

2: Upon receiving $(\mathtt{Delivered}, \mathtt{DZK})$ from $\mathcal{F}_{\mathsf{dZK}}$, each party P_i sends $(\mathtt{VerifyDZK}, f_1(\alpha_i), \ldots, f_L(\alpha_i), \alpha_i)$ to $\mathcal{F}_{\mathsf{dZK}}$ when he receives m_i. If he receives $\mathtt{true}$ from $\mathcal{F}_{\mathsf{dZK}}$, he sends r_i to D.
- For $i \in [t]$, if P_i receives $(\mathtt{Init}, \mathsf{key}_i)$ from $\mathcal{F}^{(i)}_{\mathsf{PrivSend\text{-}Ab}}$, he locally generates m_i from $H(\mathsf{key}_i)$. If he receives $(\mathtt{Init}, \mathtt{abort})$ instead, he will set his output as $\mathtt{abort}$ and send $\mathtt{abort}$ to D.

3: D initializes a set $\mathcal{M} = \emptyset$, upon receiving $H(r_i)$ and correct opening r_i from P_i, D adds (P_i, r_i) to $\mathcal{M}$. If D first receives $\mathtt{abort}$ before $|\mathcal{M}| = 2t+1$, he reliably broadcasts $\mathtt{Fail}$ and terminates. Otherwise, when $|\mathcal{M}| = 2t+1$, D does the following.
- Reliably broadcast $\mathcal{M}$.
- For each $P_i \notin \mathcal{M}$, send $(m_i, \mathtt{ACSS})$ to $\mathcal{F}^{(i)}_{\mathsf{PrivSend\text{-}Ab}}$.

Termination Phase

1: All parties wait to receive a broadcast message from D:
- If the message is $\mathtt{Fail}$, all parties terminate with $\mathtt{Fail}$.
- If the message is a set $\mathcal{M}$, all parties accept it when they receive $H(r_i)$ broadcast by P_i for each $(P_i, r_i) \in \mathcal{M}$, and then they proceed.

2: Upon receiving $(\mathtt{Delivered}, \mathtt{ACSS})$ from $\mathcal{F}^{(i)}_{\mathsf{PrivSend\text{-}Ab}}$ for all $P_i \notin \mathcal{M}$, each party P_i does the following to determine his output. Note that each party $P_i \notin \mathcal{M}$ will receives $(m_i, \mathtt{ACSS})$ or $\mathtt{abort}$ from $\mathcal{F}^{(i)}_{\mathsf{PrivSend\text{-}Ab}}$.
- If he gets his shares, he sends $(\mathtt{VerifyDZK}, f_1(\alpha_i), \ldots, f_L(\alpha_i), \alpha_i)$ to $\mathcal{F}_{\mathsf{dZK}}$, then terminates with his shares if he gets $\mathtt{true}$ from $\mathcal{F}_{\mathsf{dZK}}$ and $\mathtt{abort}$ otherwise.
- Otherwise, he terminates with $\mathtt{abort}$.

Public Reconstruction Phase

1: Each party $P_i \in \mathcal{M}$ sends $\langle Shares, f_1(\alpha_i), \ldots, f_L(\alpha_i)\rangle$ to all parties. When P_i receives $\langle Shares\rangle$ from P_j, he follows the Verification Phase to check P_j's shares. When P_i receives correct $\langle Shares\rangle$ from $t+1$ distinct P_j, he reconstructs and outputs $f_1(x), \ldots, f_L(x)$.

LEMMA 5.1. *Protocol $\Pi_{\mathsf{ACSS\text{-}Ab}}$ securely computes $\mathcal{F}_{\mathsf{ACSS\text{-}Ab}}$ in the $\{\mathcal{F}_{\mathsf{dZK}}, \mathcal{F}_{\mathsf{PrivSend\text{-}Ab}}\}$-hybrid model against a fully malicious adversary $\mathcal{A}$ who corrupts at most $t < n/3$ parties.*

Protocol $\Pi_{\mathsf{Sh2t\text{-}Ab}}$

Let $\alpha_0, \ldots, \alpha_n$ be distinct field elements, L be the number of sharings to be prepared. Denote $\mathcal{F}^{(i)}_{\mathsf{PrivSend\text{-}Ab}}$, key_i as the instance of $\mathcal{F}_{\mathsf{PrivSend\text{-}Ab}}$ and the symmetric key between the dealer D and each P_i.

Distributing Phase

1: D possesses secret $s_1, \ldots, s_L$ as inputs. For $i \in [2t]$, D expends $H(\mathsf{key}_i)$ to get L random values, denoted by $\{h^{(i)}_\ell\}^L_{\ell=1}$. Then for each $\ell \in [L]$, D computes a degree-$2t$ polynomial $f_\ell(\cdot)$ such that $f_\ell(\alpha_0) = s_\ell$ and $f_\ell(\alpha_i) = h^{(i)}_\ell$.

2: For $i \in [2t+1, n]$, let $m_i = \langle Shares, f_1(\alpha_i), \ldots, f_L(\alpha_i)\rangle$, D sends $(m_i, \mathtt{Sh2t})$ to $\mathcal{F}^{(i)}_{\mathsf{PrivSend\text{-}Ab}}$.

Termination Phase

1: When all parties receive $(\mathtt{Delivered}, \mathtt{Sh2t})$ from $\mathcal{F}^{(i)}_{\mathsf{PrivSend\text{-}Ab}}$ for all $i \in [2t+1, n]$, they proceed.

2: For each $i \in [2t]$, if P_i receives $(\mathtt{Init}, \mathsf{key}_i)$ from $\mathcal{F}^{(i)}_{\mathsf{PrivSend\text{-}Ab}}$, he locally generates m_i from $H(\mathsf{key}_i)$. If he receives $(\mathtt{Init}, \mathtt{abort})$ instead, he will set his output as $\mathtt{abort}$ and terminate.

3: For each $i \in [2t+1, n]$, if P_i receives $(m_i, \mathtt{Sh2t})$ from $\mathcal{F}^{(i)}_{\mathsf{PrivSend\text{-}Ab}}$, he terminates with his shares. Otherwise, if he receives $\mathtt{abort}$ instead, he terminates with $\mathtt{abort}$.

LEMMA 5.2. *Protocol $\Pi_{\mathsf{Sh2t\text{-}Ab}}$ securely computes $\mathcal{F}_{\mathsf{Sh2t\text{-}Ab}}$ in the $\mathcal{F}_{\mathsf{PrivSend\text{-}Ab}}$-hybrid model against a fully malicious adversary $\mathcal{A}$ who corrupts at most $t < n/3$ parties.*

We prove Lemma 5.1, Lemma 5.2 and analyze the costs in Appendix D.1 and Appendix D.2, respectively.

6 PREPARATION OF RANDOM SHARINGS

In this section, we give our construction of random sharing generation protocols. They achieve an amortized linear cost per random sharing, along with $O(n^3)$ additive overhead.

6.1 Functionality of Preparing Random Degree-t and $2t$ Sharings

$\mathcal{F}_{\mathsf{RandSh\text{-}Ab}}, \mathcal{F}_{\mathsf{RandShZero\text{-}Ab}}$ corresponds to the functionalities of generating degree-t and $2t$ random sharings, respectively. For $\mathcal{F}_{\mathsf{RandSh\text{-}Ab}}$, it supports public reconstruction of the random secrets when all parties successfully terminate the generation phase. For $\mathcal{F}_{\mathsf{RandShZero\text{-}Ab}}$, it allows the adversary to insert an additive error into each party's output shares.

Functionality $\mathcal{F}_{\mathsf{RandSh\text{-}Ab}}$

Public Input: N

$\mathcal{F}_{\mathsf{RandSh\text{-}Ab}}$ proceeds as follows, running with parties $\mathcal{P} = \{P_1, \ldots, P_n\}$ and an adversary $\mathcal{S}$.

1: For all $\ell \in [N]$, the trusted party randomly samples r_ℓ.

2: For all $\ell \in [N]$, the trusted party receives a set of shares of corrupted parties from $\mathcal{S}$ and samples a random degree-t Shamir sharing $[r_\ell]_t$ based on the shares of corrupted parties and the secret r_ℓ.

3: For all $\ell \in [N]$ and each party P_j, send a request-based delayed output of the jth value of $[r_\ell]_t$ to P_j.

- Upon receiving a request $(\texttt{abort}, P_j)$ from $\mathcal{S}$, if the output of P_j has not been delivered, change the output of P_j to `abort`. Otherwise, ignore this request.
- Upon receiving a request `Fail` from $\mathcal{S}$, if the output of all parties has not been delivered, change the output of all parties to `Fail`. Otherwise, ignore this request.

4: Upon receiving request `Public-Recon` from $t+1$ distinct parties, if it has not received `Fail` from $\mathcal{S}$ before all parties' output are delivered, it sends the whole sharing $([r_1]_t, \ldots, [r_N]_t)$ to all parties.

Functionality $\mathcal{F}_{\text{RandShZero-Ab}}$

Public Input: N

$\mathcal{F}_{\text{RandShZero-Ab}}$ proceeds as follows, running with parties $\mathcal{P} = \{P_1, \ldots, P_n\}$ and an adversary $\mathcal{S}$.

1: For all $\ell \in [N]$, the trusted party receives a set of shares of corrupted parties and an additive error vector $\boldsymbol{e}_\ell$ from $\mathcal{S}$. Then it samples a random degree-$2t$ Shamir sharing $[o_\ell]_{2t}$ based on the shares of corrupted parties and the secret $o_\ell = 0$.

2: For all $\ell \in [N]$, denote the jth value of $[o_\ell]_{2t}, \boldsymbol{e}_\ell$ as $o_\ell^{(j)}, e_\ell^{(j)}$, respectively, then it sends a request-based delayed output $o_\ell^{(j)} + e_\ell^{(j)}$ to P_j.

- Upon receiving a request $(\texttt{abort}, P_j)$ from $\mathcal{S}$, if the output of P_j has not been delivered, change the output of P_j to `abort`.

6.2 Construction of Preparing Random Degree-t and $2t$ Sharings

The following protocols $\Pi_{\text{RandSh-Ab}}, \Pi_{\text{RandShZero-Ab}}$ realize $\mathcal{F}_{\text{RandSh-Ab}}$, $\mathcal{F}_{\text{RandShZero-Ab}}$, respectively. To guarantee the public reconstruction of secrets, after all parties agree on a set of successful dealers, they check whether they do not terminate with `Fail` for each $\mathcal{F}_{\text{ACSS-Ab}}$ invoked by dealers in this set. If true, they can later reconstruct each dealer's secrets and compute the linear combination to get the secrets of random degree-t sharings.

Protocol $\Pi_{\text{RandSh-Ab}}$

Let N be the number of random degree-t Shamir secret sharings to be prepared.

Generation Phase

1: Each party P_i samples $N' = N/(t+1)$ random values $s_1^{(i)}, \ldots, s_{N'}^{(i)}$. Then he acts as the dealer D, invoking $\mathcal{F}_{\text{ACSS-Ab}}$ to distribute these values to all parties.

2: Each party P_i sets the property Q as P_i terminating $\mathcal{F}_{\text{ACSS-Ab}}$ when P_j acts as a dealer. Then all parties invoke $\mathcal{F}_{\text{acs}}$ with property Q to agree on a set $\mathcal{D}$ of successful dealers with size $|\mathcal{D}| = 2t+1$.

3: Each party waits to receive his output from $\mathcal{F}_{\text{ACSS-Ab}}$ for each dealer $P_i \in \mathcal{D}$, if his output is `abort` or `Fail`, he terminates with it; otherwise, he proceeds.

4: All parties agree on (the inverse of) a Vandermonde matrix $\boldsymbol{M}$ of size $(t+1) \times (2t+1)$. Denote the sharing distributed by each party P_i as $([s_1^{(i)}]_t, \ldots, [s_{N'}^{(i)}]_t)$. For all $\ell \in [N']$, each party computes his shares by following

$$([r_{\ell,1}]_t, \ldots, [r_{\ell,t+1}]_t) = \boldsymbol{M} \cdot ([s_\ell^{(i)}]_t)_{i \in \mathcal{D}}.$$

Finally, he outputs his shares of $\{[r_{\ell,i}]_t\}_{\ell \in [N'], i \in [t+1]}$ and terminates.

Upon receiving the request `Public-Recon` from the environment, all parties do the following.

Public Reconstruction Phase

1: For each party, he proceeds to the next step if his output of the generation phase is not `Fail`.

2: For each dealer $P_i \in \mathcal{D}$, all parties send request `Public-Recon` to $\mathcal{F}_{\text{ACSS-Ab}}$ led by P_i and learns the whole sharing $[s_1^{(i)}]_t, \ldots, [s_N^{(i)}]_t$. Then all parties locally extract the random values $\{r_{\ell,i}\}_{\ell \in [N'], i \in [t+1]}$ and terminates.

LEMMA 6.1. *Protocol $\Pi_{\text{RandSh-Ab}}$ securely computes $\mathcal{F}_{\text{RandSh-Ab}}$ in the $\{\mathcal{F}_{\text{ACSS-Ab}}, \mathcal{F}_{\text{acs}}\}$-hybrid model against a fully malicious adversary $\mathcal{A}$ who corrupts at most $t < n/3$ parties.*

Protocol $\Pi_{\text{RandShZero-Ab}}$

Let N be the number of total degree-$2t$ Shamir sharings of 0. The construction is the same as $\Pi_{\text{RandSh-Ab}}$, except that there is no public reconstruction phase and in Step 4 of generation phase, denote the sharing distributed by each party P_i as $([o_1^{(i)}]_{2t}, \ldots, [o_{N'}^{(i)}]_{2t})$, and for all $\ell \in [N']$, all parties locally compute

$$([o_{\ell,1}]_{2t}, \ldots, [o_{\ell,t+1}]_{2t}) = \boldsymbol{M} \cdot ([o_\ell^{(i)}]_{2t})_{i \in \mathcal{D}}.$$

LEMMA 6.2. *Protocol $\Pi_{\text{RandShZero-Ab}}$ securely computes functionality $\mathcal{F}_{\text{RandShZero-Ab}}$ in the $\{\mathcal{F}_{\text{Sh2t-Ab}}, \mathcal{F}_{\text{acs}}\}$-hybrid model against a fully malicious adversary $\mathcal{A}$ who corrupts at most $t < n/3$ parties.*

We prove Lemma 6.1, Lemma 6.2 and analyze the costs in Appendix D.3 and Appendix D.4, respectively.

Generating Random Shamir Sharings in Pipeline. As we mentioned in section 2.2, the pipeline technique can be used to optimize the constant factor by 1/3. Refer to Appendix D.5 for detailed construction.

7 MAIN PROTOCOL

In this section, we give the construction of our protocol Velox, an AMPC protocol with fairness. We start with the construction of the efficient DN multiplication protocol introduced in section 2.3, then give the circuit evaluation protocol based on the ideas introduced in Appendix B.2, and finally the whole AMPC protocol.

7.1 Efficient Multiplication Protocol

Refer to section 2.3 for the ideas about the following construction. The achieved communication complexity is amortized linear per multiplication gate, along with $O(n^2)$ additive overhead.

Protocol $\Pi_{\texttt{Mult}}$

Let N be the number of inner products that need to be computed and $\alpha_1, \ldots, \alpha_n$ be distinct field elements, all parties take their shares of $\{[a^{(\ell)}]_t, [b^{(\ell)}]_t\}_{\ell=1}^N$ and $\{[r^{(\ell)}]_t\}_{\ell=1}^N, \{[o^{(\ell)}]_{2t}\}_{\ell=1}^{N/2}$ or $\perp$ as inputs.

Computing Multiplication

1: Divide these vectors of Shamir sharings into $N/(2t+1)$ groups, each of size $2t+1$ and denoted by $\{[\boldsymbol{a}_k]_t, [\boldsymbol{b}_k]_t, [r_k]_t\}_{k=1}^{2t+1}, \{[o_k]_{2t}\}_{k=1}^{t}$.

2: Let $\boldsymbol{M}$ be a Vandermonde matrix of size $n \times t$, for each group, all parties locally compute

$$([\tilde{o}_1]_{2t}, \ldots, [\tilde{o}_n]_{2t}) = \boldsymbol{M} \cdot ([o_1]_{2t}, \ldots, [o_t]_{2t}).$$

3: All parties locally set a degree-$2t$ polynomial $f(\cdot) \in \mathbb{F}[X]$ by using $z_k = a_k \cdot b_k + r_k$, i.e.,

$$f(X) = z_1 + z_2 \cdot X + \cdots + z_{2t+1} \cdot X^{2t}.$$

4: Each party whose input sharings are not $\perp$ first locally compute his share of $[z_k]_{2t} = [a_k]_t \cdot [b_k]_t + [r_k]_t$ for all $k \in [2t+1]$ and then $[f(\alpha_1)]_{2t}, \ldots, [f(\alpha_n)]_{2t}$.

5: All parties send their shares of $[\tilde{f}(\alpha_i)]_{2t} := [f(\alpha_i)]_{2t} + [\tilde{o}_i]_{2t}$ to P_i. Then P_i waits to receive messages from all parties:
- If P_i first receives $\perp$, P_i sets the reconstruction results as $\perp$.
- Otherwise, if P_i first receives $2t+1$ shares, P_i reconstructs $\tilde{f}(\alpha_i)$.

6: For all groups, each party P_i sends his reconstruction result $\tilde{f}(\alpha_i)$ or $\perp$ to all parties. Then P_i waits to receive messages from all parties:
- If P_i first receives $\perp$, P_i sets $Z^{(i)} = \perp$.
- Otherwise, if P_i first receives $\tilde{f}(\alpha_j)$ from $2t+1$ distinct P_j, P_i reconstructs $\tilde{f}(x)$ and gets the corresponding coefficients, denoted by $\{\tilde{e}_k\}_{k=1}^{2t+1}$. Then P_i sets $Z^{(i)}$ as the hash of $\{\tilde{z}_k\}_{k=1}^{2t+1}$ for all groups.

Finally, P_i sends $Z^{(i)}$ to all parties.

Agreement of Reconstruction Result

1: Upon receiving $Z^{(j)}$ from $2t+1$ distinct P_j, if any of them is $\perp$ or $Z^{(j)} \neq Z^{(i)}$, P_i outputs `Fail`. Otherwise, he continues.

2: If a party's input sharings are $\perp$, he outputs `Fail`. Otherwise, he locally computes $[c_k]_t = \tilde{z}_k - [r_k]_t$ and outputs his shares of $\{[c_k]_t\}$ for all $k \in [2t+1]$ and groups.

7.2 Evaluating a Two-Layer Circuit in Parallel

We combine our multiplication protocol $\Pi_{\texttt{Mult}}$ and the techniques in [GLO+21] to construct a circuit evaluation protocol $\Pi_{\texttt{Eval}}$. It allows all parties to evaluate a two-layer circuit in parallel, and this will help to reduce the round complexity in the online phase by a factor of 2. Refer to Appendix B.2 for the high-level ideas and the detailed construction.

7.3 Construction of the Fairness AMPC

To first construct a security-with-abort AMPC, we can first use $\mathcal{F}_{\texttt{RandSh-Ab}}, \mathcal{F}_{\texttt{RandShZero-Ab}}$ to prepare random degree-t and $2t$ in the offline phase, then all parties invoke $\Pi_{\texttt{Eval}}$ to evaluate the circuit in the online phase. Let $|C|$ denote the size of circuit and D denote the circuit depth, the communication complexity is $O(|C| \cdot n + D \cdot n^2 + n^3)$.

To upgrade it to achieve fairness, as we mentioned in section 2.4, we need to let all parties prepare random degree-t sharings as masks and guarantee the public reconstruction when they need. To achieve this, we note that $\mathcal{F}_{\texttt{RandSh-Ab}}$ is already sufficient to achieve this goal. If all parties do not terminate $\mathcal{F}_{\texttt{RandSh-Ab}}$ with `Fail`, they can guarantee the public reconstruction of random secrets later. Otherwise, they can abort the protocol and the adversary still learn nothing about the output. Let C_O be the output size, this communication complexity is $O(C_O \cdot n^2 + n^3)$.

By combining them, we get our fairness AMPC, refer to Appendix F for detailed construction, security and costs analysis.

8 EXPERIMENTS AND EVALUATION

We describe the implementation and evaluation of Velox in this section, and compare it against prior works.

Implementation. We implemented Velox from end to end in Rust[6]. We used the `tokio` library as the asynchronous runtime and multithreading engine. We utilize a Fast Fourier Transform friendly 252-bit prime field. We realize finite-field arithmetic using the Lambdaworks library and utilize Fast-Fourier transforms for share preparation wherever possible. We utilize the `SHA256` hash function for hash operations and hardware-accelerated `AES256` block cipher in CBC mode for symmetric key encryption.

We also implemented the various building blocks of our protocol. We implemented Cachin-Tessaro's RBC protocol [CT05] and a batched version of DispersedLedger's AVID protocol [YPA+22]. Both these protocols use Merkle trees for generating vector commitments and Reed-Solomon Erasure codes for coding data. Our choice is influenced by a highly efficient erasure coding library in Rust, even considering the $O(\log(n))$ factor increase in the additive communication overhead. Furthermore, we implemented Das *et al.*'s ACS protocol [DDL+24], which only requires Hash functions and a secure channel setup. Our implementation does not make use of the pipeline optimization as described in section 2.2. We followed a modular design and implemented each building block of Velox as a separate standalone library. These building blocks can be used and composed in a plug-and-play manner to develop more advanced protocols and, therefore, can be of independent interest.

Commitments for Long Messages We employ a cryptographic Hash function for Random Oracle invocations. In Velox, we invoke the Hash function to generate commitments on long messages of length $O(L)$. However, the computational cost of a Hash function like `SHA256` increases linearly with input length. In addition, CPU-based implementation of `SHA256` is slow, with a 10 KB message taking 38 microseconds for Hashing, as compared to 500 nanoseconds for a 512 byte message.

We reduce this cost by employing a Hash function built using the AES symmetric-key cipher [RS08]. The main advantage of this function is that we can employ hardware-accelerated variant of AES for commitment generation, available as the AES-NI instruction set in contemporary processors. However, this construction only offers a fixed compression ratio of 2 i.e. it compresses an input of size 512 bits to 256 bits. So, to hash a long message, we create a Merkle tree of this message and compute the root of this tree as the message's Hash. This construction takes only takes 11 microseconds to hash a 10 KB message, resulting in a 3.4× speedup compared to CPU-based `SHA256`.

[6] Available at https://github.com/akhilsb/Velox-MPC

Application. We implement anonymous broadcast using Velox. An anonymous broadcast is a crucial tool in anonymous communication that enables a set of clients to broadcast information anonymously. We implement the AsynchroMix protocol in HoneyBadgerMPC [LYK+19]. In this protocol, k clients each want to broadcast a message $m_i : i \in \{1, .., k\}$ to a bulletin board, visible and available to the outside world. Here, k is the size of the anonymity set. A set of n parties run a mixing service or mixer using our MPC protocol. Essentially, the clients use $\mathcal{F}_{\text{ACSS-Ab}}$ to share their message to the mixer. On terminating ACSS for all k clients, parties running the mixer execute a switching network circuit based on the AsynchroMix circuit in HoneyBadgerMPC.

This circuit is of depth $\log_2^2(k)$ and contains $\frac{k}{2}$ multiplication gates at each depth. Looking ahead, each set of $\log_2(k)$ depths constitute a butterfly network pattern. In each depth $d \in [0, \log_2^2(k)]$, each multiplication gate consists of $\frac{k}{2}$ multiplication gates, each gate responsible for randomly shuffling a pair of input messages $(m_{i1,d}, m_{i2,d})$. In further detail, each gate at a depth d multiplies $[b]_t$, which is a random sharing of +1 or −1, and $[m_{i1} - m_{i2}]$. Then, the result $M_{i1,i2,d}$ is added to and subtracted from $[m_{i1,d} + m_{i2,d}]_t$ to generate shares for next gate $m_{i1,d+1} = 2^{-1} \cdot ([m_{i1,d} + m_{i2,d}]_t + [M_{i1,i2,d}]_t)$ and $m_{i2,d+1} = 2^{-1} \cdot ([m_{i1,d} + m_{i2,d}]_t - [M_{i1,i2,d}]_t)$. Across depths, the pairs are formed based on a butterfly network pattern, where the i-th input wire at depth d is paired with the wire at index $2^{\log_2(k) - d\%\log_2(k)} + i$.

This circuit requires preparing sharings of random bits $(+1/-1)$. We follow HoneyBadgerMPC's approach for this, where parties utilize replicated secret sharing to generate pairs of identical random sharings. This preparation requires one multiplication and one reconstruction operation per random bit sharing. Therefore, for a mix circuit with k input messages, we prepare $k \cdot \log_2^2(k)$ random double sharings, each for one multiplication.

Comparison with prior works. We compare Velox with DumboMPC[SLL+24], the current best work on asynchronous MPC. This protocol generates Beaver multiplication triples that can be used to evaluate multiplication gates in a future online phase. DumboMPC also provides two different protocols `OptRanTriGen`, an *optimistic* variant that works when no party behaves maliciously, and `AsyRanTriGen`, a *pessimistic* variant that works with multiple faulty parties. We compare the triple-generation latency of optimistic `OptRanTriGen` in DumboMPC with our end-to-end execution time. In further detail, we estimate the number of multiplication triples required by `OptRanTriGen` to evaluate a mixing circuit for a given anonymity set size k and compute the amount of time taken to generate these triples. Based on our described estimate, we set this number as $k \log_2^2(k)$ triples. We also noticed that the code of DumboMPC uses only a single CPU thread to generate triples. We offset this problem by multiplying its numbers by the number of CPU cores available to each party and comparing these amplified numbers to our numbers.

Evaluation setup. We evaluate the prior works and our work in a Local Area Network (LAN) testbed on Amazon Web Services (AWS). We choose `c5.large` instances with 2 CPU cores and 4 GB RAM in `us-east-1` (N. Virginia). We run the protocol with a varying number of parties $n = 16, 64, 112$ and varying anonymity set sizes $k = 256, 512, 1024$ messages.

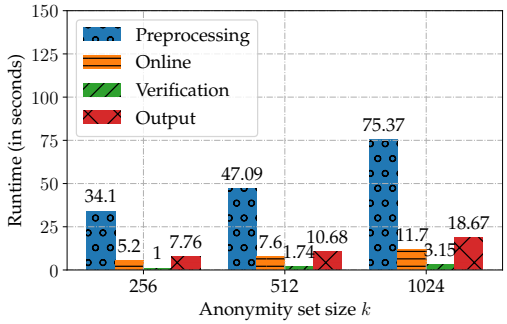


Figure 1: Phase-wise runtime: We plot a phase-wise latency of our protocol Velox running with $n = 112$ parties mixing a batch of $k = 256, 512, 1024$ messages. The preprocessing phase, where parties generate random sharings, is the most expensive phase of all. The output phase, where the random masks need to be reconstructed, also becomes more expensive with increasing output size because of quadratic communication cost.

Configuration. We configured Velox by varying the compression factor k_{opt} in the verification phase, which balances computation and round complexity (see Appendix E). We found that setting $k_{\text{opt}} = 10$ minimizes the verification runtime, although the optimal value may vary depending on the tradeoff.

Additionally, we introduced a threshold-based switch between our multiplication protocols with linear and quadratic communication per depth. While the linear variant reduces per-gate communication, it incurs one additional round. We set the threshold at 10 multiplication gates per depth: if the number of multiplications is below this threshold, the protocol switches to the quadratic variant; otherwise, it uses the linear one.

Results. We first plot the phase-wise runtime of Velox with increasing anonymity set size k in Fig. 1 at $n = 112$ parties. Velox can be divided into the preprocessing phase where we prepare random double sharings, the online phase with circuit evaluation, the verification phase with multiplication tuple verification, and the output phase with random mask reconstruction. The preprocessing phase accounts for the majority of runtime in all three cases. This is mainly due to the computational cost of generating and validating secret shares. This cost can be shifted to the offline phase by starting preprocessing much before clients start the protocol. In the online phase, the output reconstruction phase becomes increasingly expensive as k increases. This is because of the quadratic communication cost incurred for reconstructing each random mask.

We also measure the end-to-end runtime (offline and online) of Velox and preprocessing phase of DumboMPC with increasing anonymity set size $k = 256, 512, 1024$ in Fig. 2. These circuits have $32768, 82944, 204800$ multiplication gates spread over $64, 81, 100$ depths, respectively. Velox achieves an order of magnitude speedup compared to DumboMPC at $n = 16, 64$ parties. DumboMPC also ran out of memory at $k = 256, 512, 1024$. We ran it in `c5.2xlarge` machines with twice the specs of our machines to generate values at $k = 256$. We were unable to generate numbers for $k = 512, 1024$.

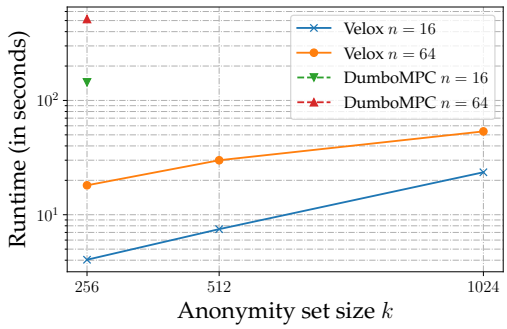


Figure 2: Anonymity set size k vs Runtime: We plot the runtime of Velox and DumboMPC's `OptRanTriGen` protocol at $n = 16, 64$ parties. Velox is $36\times$ faster $n = 16$ and $28.6\times$ faster than DumboMPC at $n = 64$ for $k = 256$. We were unable to successfully run DumboMPC for higher values of k.

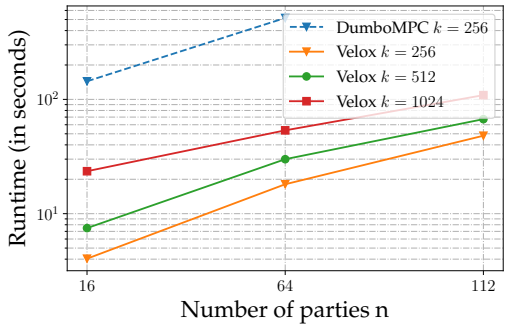


Figure 3: Scalability plot: We plot the runtime of Velox and the `OptRanTriGen` protocol in DumboMPC at $n = 16, 64, 112$ and anonymity set sizes $k = 256, 512, 1024$. Velox is able to mix a batch of $k = 256, 512, 1024$ messages within 48, 67, and 108 seconds with $n = 112$ parties, respectively.

Towards real-time MPC At $n = 16$ parties, Velox is able to mix $k = 256, 512, 1024$ messages within 4, 7.4, and 22.1 seconds, respectively. However, on running the protocol in more powerful `c5.4xlarge` machines with 16 CPU cores, 32 GB RAM, and $n = 16$ parties, this runtime reduces to 1.3, 2.4, and 5.6 seconds for $k = 256, 512, 1024$ messages, respectively. This improvement mainly comes from reduced computation time in the preprocessing phase, which is a result of the increased number of cores. To be more concrete, Velox processes over 70000 multiplication gates per second at $n = 16$ in these machines. Hence, our techniques enable Velox to make real-time or sub-second MPC possible, further improving its practicality in various applications.

Scalability. We also evaluated Velox at $n = 4, 16, 64, 112$ parties and $k = 256, 512, 1024$ messages in Fig. 3. Essentially, at $n = 112$, Velox takes under 1 and 2 minutes to anonymously broadcast a set of $k = 256, 1024$ messages, respectively. In comparison, DumboMPC requires 515 seconds to generate the required number of triples for the $k = 256$ circuit at $n = 64$, which indicates $28.6\times$ speedup for Velox.

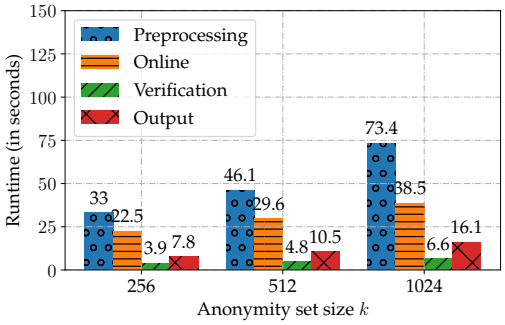


Figure 4: Phase-wise runtime in a geo-distributed testbed: We plot a phase-wise latency of Velox running with $n = 112$ parties mixing a batch of $k = 256, 512, 1024$ messages. The preprocessing phase, where parties generate random sharings, is the most expensive phase of all. In this setting with a high round trip time, the online phase contributes a substantial latency to the overall runtime, mainly because of its round complexity.

Evaluation in a geo-distributed testbed We also evaluated Velox in a geo-distributed AWS testbed. This testbed comprises of `c5.large` machines from 8 AWS regions across the world: Virginia, Ohio, N. California, Oregon, Canada, Ireland, Singapore, and Tokyo. This setting has a much higher round trip time of 100 milliseconds compared to <1 millisecond in the LAN setting. We describe the phase-wise runtime decomposition of Velox's runtime at $n = 112$ parties in Fig. 4. The online phase's latency for $k = 256, 512, 1024$ increases by 17, 22, 27 seconds compared to the LAN setting. This increase is mainly because of the $O(\log_2^2(k))$ round complexity of the online phase and the high round trip time.

We also plot the scalability of Velox at $n = 16, 64, 112$ parties in Fig. 5. For $k = 256$, Velox takes 27, 23, 19 seconds more time than the LAN setting at $n = 16, 64, 112$ parties, indicating the constant overhead of $\log_2^2(k)$ rounds in the online phase. At $k = 1024$, these numbers increase to being 39, 36, 28 seconds higher than the LAN setting for $n = 16, 64, 112$ parties. These numbers can be improved by designing more round-efficient circuits for such applications in the geo-distributed setting. We leave this problem as a prospective future work.

REFERENCES

[AAPP24] Ittai Abraham, Gilad Asharov, Shravani Patil, and Arpita Patra. Perfect asynchronous MPC with linear communication overhead. In Marc Joye and Gregor Leander, editors, *EUROCRYPT 2024, Part V*, volume 14655 of *LNCS*, pages 280–309. Springer, Cham, May 2024.

[ABCP23] Shahla Atapoor, Karim Baghery, Daniele Cozzo, and Robi Pedersen. VSS from distributed ZK proofs and applications. In Jian Guo and Ron Steinfeld, editors, *ASIACRYPT 2023, Part I*, volume 14438 of *LNCS*, pages 405–440. Springer, Singapore, December 2023.

[ADS20] Ittai Abraham, Danny Dolev, and Gilad Stern. Revisiting asynchronous fault tolerant computation with optimal resilience. In Yuval Emek and Christian Cachin, editors, *39th ACM PODC*, pages 139–148. ACM, August 2020.

[BBB+24] Akhil Bandarupalli, Adithya Bhat, Saurabh Bagchi, Aniket Kate, and Michael K. Reiter. Random beacons in monte carlo: Efficient asynchronous random beacon without threshold cryptography. ACM CCS 2024, 2024. https://eprint.iacr.org/2023/1755.

[BCG93] Michael Ben-Or, Ran Canetti, and Oded Goldreich. Asynchronous secure computation. In *25th ACM STOC*, pages 52–61. ACM Press, May 1993.

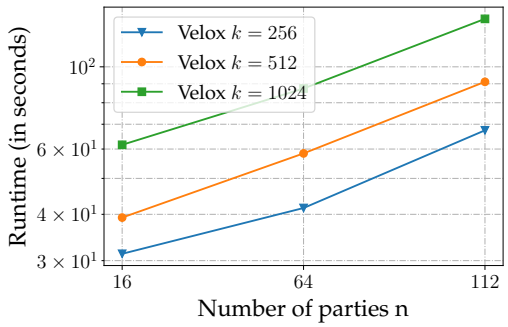


Figure 5: Scalability plot in geo-distributed testbed: We plot the runtime of Velox at $n = 16, 64, 112$ and anonymity set sizes $k = 256, 512, 1024$. At $n = 112$, Velox takes $67, 91, 134$ seconds to anonymously broadcast a set of $k = 256, 512, 1024$ messages, respectively.

[Bea92] Donald Beaver. Efficient multiparty protocols using circuit randomization. In Joan Feigenbaum, editor, *Advances in Cryptology — CRYPTO '91*, volume 576 of *Lecture Notes in Computer Science*, pages 420–432, Berlin, Heidelberg, 1992. Springer Berlin Heidelberg.

[BGW88] Michael Ben-Or, Shafi Goldwasser, and Avi Wigderson. Completeness theorems for non-cryptographic fault-tolerant distributed computation (extended abstract). In *20th ACM STOC*, pages 1–10. ACM Press, May 1988.

[BJK+25] Akhil Bandarupalli, Xiaoyu Ji, Aniket Kate, Chen-Da Liu-Zhang, and Yifan Song. Computationally efficient asynchronous mpc with linear communication and low additive overhead. In CRYPTO 2025, 2025.

[BKLZL20] Erica Blum, Jonathan Katz, Chen-Da Liu-Zhang, and Julian Loss. Asynchronous byzantine agreement with subquadratic communication. In Rafael Pass and Krzysztof Pietrzak, editors, *TCC 2020, Part I*, volume 12550 of *LNCS*, pages 353–380. Springer, Cham, November 2020.

[BKR94] Michael Ben-Or, Boaz Kelmer, and Tal Rabin. Asynchronous secure computations with optimal resilience (extended abstract). In Jim Anderson and Sam Toueg, editors, *13th ACM PODC*, pages 183–192. ACM, August 1994.

[BOKR94] Michael Ben-Or, Boaz Kelmer, and Tal Rabin. Asynchronous secure computations with optimal resilience (extended abstract). In *Proceedings of the Thirteenth Annual ACM Symposium on Principles of Distributed Computing*, PODC '94, page 183–192, New York, NY, USA, 1994. Association for Computing Machinery.

[Can00] Ran Canetti. Security and composition of multiparty cryptographic protocols. *Journal of Cryptology*, 13:143–202, 2000.

[CCD88] David Chaum, Claude Crépeau, and Ivan Damgård. Multiparty unconditionally secure protocols (extended abstract). In *20th ACM STOC*, pages 11–19. ACM Press, May 1988.

[CGHZ16] Sandro Coretti, Juan A. Garay, Martin Hirt, and Vassilis Zikas. Constant-round asynchronous multi-party computation based on one-way functions. In Jung Hee Cheon and Tsuyoshi Takagi, editors, *ASIACRYPT 2016, Part II*, volume 10032 of *LNCS*, pages 998–1021. Springer, Berlin, Heidelberg, December 2016.

[CHLZ21] Annick Chopard, Martin Hirt, and Chen-Da Liu-Zhang. On communication-efficient asynchronous MPC with adaptive security. In Kobbi Nissim and Brent Waters, editors, *TCC 2021, Part II*, volume 13043 of *LNCS*, pages 35–65. Springer, Cham, November 2021.

[Coh16] Ran Cohen. Asynchronous secure multiparty computation in constant time. In Chen-Mou Cheng, Kai-Min Chung, Giuseppe Persiano, and Bo-Yin Yang, editors, *PKC 2016, Part II*, volume 9615 of *LNCS*, pages 183–207. Springer, Berlin, Heidelberg, March 2016.

[CP15] Ashish Choudhury and Arpita Patra. Optimally resilient asynchronous MPC with linear communication complexity. In *Proc. Intl. Conference on Distributed Computing and Networking (ICDCN)*, pages 1–10, 2015.

[CP23] Ashish Choudhury and Arpita Patra. On the communication efficiency of statistically secure asynchronous mpc with optimal resilience. *Journal of Cryptology*, 36(2):13, 2023.

[CR98] Ran Canetti and Tal Rabin. Fast asynchronous byzantine agreement with optimal resilience, 1998.

[CT05] C. Cachin and S. Tessaro. Asynchronous verifiable information dispersal. In *24th IEEE SRDS'05*, pages 191–201, 2005.

[DDL+24] Sourav Das, Sisi Duan, Shengqi Liu, Atsuki Momose, Ling Ren, and Victor Shoup. Asynchronous consensus without trusted setup or public-key cryptography. *Cryptology ePrint Archive*, 2024.

[DGKN09] Ivan Damgård, Martin Geisler, Mikkel Krøigaard, and Jesper Buus Nielsen. Asynchronous multiparty computation: Theory and implementation. In Stanislaw Jarecki and Gene Tsudik, editors, *PKC 2009*, volume 5443 of *LNCS*, pages 160–179. Springer, Berlin, Heidelberg, March 2009.

[DN07] Ivan Damgård and Jesper Buus Nielsen. Scalable and unconditionally secure multiparty computation. In *Advances in Cryptology - CRYPTO 2007, 27th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 19-23, 2007, Proceedings*, volume 4622 of *Lecture Notes in Computer Science*, pages 572–590. Springer, 2007.

[DR85] Danny Dolev and Rüdiger Reischuk. Bounds on information exchange for byzantine agreement. *Journal of the ACM (JACM)*, 32(1):191–204, 1985.

[DXR21] Sourav Das, Zhuolun Xiang, and Ling Ren. Asynchronous data dissemination and its applications. In Giovanni Vigna and Elaine Shi, editors, *ACM CCS 2021*, pages 2705–2721. ACM Press, November 2021.

[GIP+14] Daniel Genkin, Yuval Ishai, Manoj M Prabhakaran, Amit Sahai, and Eran Tromer. Circuits resilient to additive attacks with applications to secure computation. In *Proceedings of the forty-sixth annual ACM symposium on Theory of computing*, pages 495–504, 2014.

[GLO+21] Vipul Goyal, Hanjun Li, Rafail Ostrovsky, Antigoni Polychroniadou, and Yifan Song. Atlas: efficient and scalable mpc in the honest majority setting. In *Advances in Cryptology–CRYPTO 2021: 41st Annual International Cryptology Conference, CRYPTO 2021, Virtual Event, August 16–20, 2021, Proceedings, Part II 41*, pages 244–274. Springer, 2021.

[GLZS24] Vipul Goyal, Chen-Da Liu-Zhang, and Yifan Song. Towards achieving asynchronous MPC with linear communication and optimal resilience. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part VIII*, volume 14927 of *LNCS*, pages 170–206. Springer, Cham, August 2024.

[GMW87] Oded Goldreich, Silvio Micali, and Avi Wigderson. How to play any mental game or A completeness theorem for protocols with honest majority. In Alfred Aho, editor, *19th ACM STOC*, pages 218–229. ACM Press, May 1987.

[GS20] Vipul Goyal and Yifan Song. Malicious security comes free in honest-majority mpc. *Cryptology ePrint Archive*, 2020.

[HNP05] Martin Hirt, Jesper Buus Nielsen, and Bartosz Przydatek. Cryptographic asynchronous multi-party computation with optimal resilience (extended abstract). In Ronald Cramer, editor, *EUROCRYPT 2005*, volume 3494 of *LNCS*, pages 322–340. Springer, Berlin, Heidelberg, May 2005.

[HNP08] Martin Hirt, Jesper Buus Nielsen, and Bartosz Przydatek. Asynchronous multi-party computation with quadratic communication. In Luca Aceto, Ivan Damgård, Leslie Ann Goldberg, Magnús M. Halldórsson, Anna Ingólfsdóttir, and Igor Walukiewicz, editors, *ICALP 2008, Part II*, volume 5126 of *LNCS*, pages 473–485. Springer, Berlin, Heidelberg, July 2008.

[JLS24] Xiaoyu Ji, Junru Li, and Yifan Song. Linear-communication asynchronous complete secret sharing with optimal resilience. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part VIII*, volume 14927 of *LNCS*, pages 418–453. Springer, Cham, August 2024.

[LYK+19] Donghang Lu, Thomas Yurek, Samarth Kulshreshtha, Rahul Govind, Aniket Kate, and Andrew K. Miller. HoneyBadgerMPC and AsynchroMix: Practical asynchronous MPC and its application to anonymous communication. In Lorenzo Cavallaro, Johannes Kinder, XiaoFeng Wang, and Jonathan Katz, editors, *ACM CCS 2019*, pages 887–903. ACM Press, November 2019.

[MMR15] Achour Mostéfaoui, Hamouma Moumen, and Michel Raynal. Signature-free asynchronous binary byzantine consensus with $t < n/3$, $o(n^2)$ messages, and o(1) expected time. *J. ACM*, 62(4), 8 2015.

[Mom24] Atsuki Momose. Practical asynchronous MPC from lightweight cryptography. Cryptology ePrint Archive, Paper 2024/1717, 2024.

[MR15] Achour Mostéfaoui and Michel Raynal. Intrusion-tolerant broadcast and agreement abstractions in the presence of byzantine processes. *IEEE Transactions on Parallel and Distributed Systems*, 27(4):1085–1098, 2015.

[PCR08] Arpita Patra, Ashish Choudhury, and C. Pandu Rangan. Efficient asynchronous multiparty computation with optimal resilience. Cryptology ePrint Archive, Report 2008/425, 2008.

[PCR10] Arpita Patra, Ashish Choudhary, and C. Pandu Rangan. Efficient statistical asynchronous verifiable secret sharing with optimal resilience. In Kaoru Kurosawa, editor, *ICITS 09*, volume 5973 of *LNCS*, pages 74–92. Springer, Berlin, Heidelberg, December 2010.

[RB89] Tal Rabin and Michael Ben-Or. Verifiable secret sharing and multiparty protocols with honest majority (extended abstract). In *21st ACM STOC*, pages 73–85. ACM Press, May 1989.

[RS08] Phillip Rogaway and John Steinberger. Constructing cryptographic hash functions from fixed-key blockciphers. In *CRYPTO*, pages 433–450. Springer, 2008.

[Sha79] Adi Shamir. How to share a secret. *Commun. ACM*, 22(11):612–613, November 1979.

[SLL+24] Yuan Su, Yuan Lu, Jiliang Li, Yuyi Wang, Chengyi Dong, and Qiang Tang. Dumbo-mpc: Efficient fully asynchronous mpc with optimal resilience. *USENIX Security 2025*, 2024.

[SS24] Victor Shoup and Nigel P. Smart. Lightweight asynchronous verifiable secret sharing with optimal resilience. *J. Cryptol.*, 37(3):27, 2024.

[Yao82] Andrew Chi-Chih Yao. Theory and applications of trapdoor functions (extended abstract). In *23rd FOCS*, pages 80–91. IEEE Computer Society Press, November 1982.

[YPA+22] Lei Yang, Seo Jin Park, Mohammad Alizadeh, Sreeram Kannan, and David Tse. {DispersedLedger}:{High-Throughput} byzantine consensus on variable bandwidth networks. In *19th USENIX NSDI 22*, pages 493–512, 2022.

A ADDITIONAL PRELIMINARIES

A.1 Definitions of Agreement Primitives

We describe functionalities for the agreement primitives, following the descriptions from [CGHZ16, Coh16].

Reliable Broadcast. We describe the functionality $\mathcal{F}_{\text{rbc}}$ for reliable broadcast. When a party P_s inputs a value v to the functionality as the sender, we will say that "P_s (reliably) broadcasts value v". Moreover, when some party P_j receives an output v in a reliable broadcast functionality with sender P_i, we will say that "P_j receives output v from P_i's reliable broadcast", and we will omit specifying the sender if the context is clear.

Functionality $\mathcal{F}_{\text{rbc}}$

$\mathcal{F}_{\text{rbc}}$ proceeds as follows, running with parties $P_1, \ldots, P_n$, where one of the parties is the sender P_s, and the adversary $\mathcal{S}$. Initialize $y = \perp$.

1: Upon receiving an input v from party P_s (the sender, or the adversary on behalf of the corrupted sender), set the output to $y = v$ and send v to the adversary.
2: Upon receiving v from the adversary, if P_s is corrupted and no party has received their output, then set $y = v$.
3: When the output is y set to be some value v, the functionality outputs y as a request-based delayed output to all parties.

Byzantine Agreement. We describe the functionality $\mathcal{F}_{\text{ba}}$ for Byzantine agreement.

Functionality $\mathcal{F}_{\text{ba}}$

$\mathcal{F}_{\text{ba}}$ proceeds as follows, running with parties $P_1, \ldots, P_n$ and the ideal adversary $\mathcal{S}$. Let $\mathcal{I} = \mathcal{H}$, where $\mathcal{H}$ is the set of honest parties. For each party P_i, initialize x_i and y_i to $\perp$. Let the message length be L.

1: Upon receiving $\mathcal{P}'$ from $\mathcal{S}$, with $|\mathcal{P}'| \leq t$, if no party has received output, then set $\mathcal{I} = \mathcal{H} \setminus \mathcal{P}'$.
2: Upon receiving a message $m \in \{0,1\}^L$ from party P_i, do as follows.
- If any party or $\mathcal{S}$ has received an output y, then ignore this message; otherwise, set $x_i = m$.
- If $x_i \neq \perp$ for every $P_i \in \mathcal{I}$, then set $y_j = y$ for every $j \in [n]$, where $y = x$ if all inputs $x_j = x$ for $P_j \in \mathcal{I}$, for some $x \neq \perp$. Otherwise, set $y = x_j$ for $P_j \notin \mathcal{H}$ with the smallest index.
- Send m to $\mathcal{S}$.

3: When the output y_i is set to be some value y, the functionality outputs y as a request-based delayed output to P_i.

Reliable Agreement. We describe the functionality $\mathcal{F}_{\text{ra}}$ for Reliable Agreement.

Functionality $\mathcal{F}_{\text{ra}}$

$\mathcal{F}_{\text{ra}}$ proceeds as follows, running with parties $P_1, \ldots, P_n$ and the ideal adversary $\mathcal{S}$. Let $\mathcal{I} = \mathcal{H}$, where $\mathcal{H}$ is the set of honest parties. For each party P_i, initialize x_i and y_i to $\perp$. Set AdvDeliver = 0.

1: Upon receiving $\mathcal{P}'$ from $\mathcal{S}$, with $|\mathcal{P}'| \leq t$, if no party has received output, then set $\mathcal{I} = \mathcal{H} \setminus \mathcal{P}'$.
2: Upon receiving a bit message m from party P_i, do as follows.
- If any party or $\mathcal{S}$ has received an output y, then ignore this message; otherwise, set $x_i = m$.
- If $x_i \neq \perp$ for every $P_i \in \mathcal{I}$, then set $y_j = y$ for every $j \in [n]$, where $y = x$ if all inputs $x_j = x$ for $P_j \in \mathcal{I}$, for some $x \neq \perp$. Otherwise, set AdvDeliver = 1.
- Send m to $\mathcal{S}$.

3: Upon receiving v from the adversary, if AdvDeliver = 1, then set $y_i = v$ for every $i \in [n]$.
4: When the output y_i is set to be some value y, the functionality outputs y as a request-based delayed output to P_i.

Agreement on a Common Subset. The agreement on a common subset (ACS) primitive allows the parties to agree on a set of at least $n - t$ parties that satisfy a certain property (a so-called ACS property).

Definition A.1. Let $\mathcal{P}$ be a set of n parties and let Q be a property that can be influenced by multiple protocols running in parallel. Every party $P_i \in \mathcal{P}$ can decide for every party $P_j \in \mathcal{P}$ based on the protocols running in parallel whether P_j satisfies the property towards P_i or not. If it does, we say P_i *likes* P_j *for* Q or simply P_i *likes* P_j if the property Q is clear from the context. We require that once a party likes another party, it cannot unlike it. Such a property Q is called an *ACS property* if for every pair of uncorrupted parties $(P_i, P_j) \in \mathcal{P}^2$ we have that P_i will eventually like P_j.

We state the traditional property-based formalization of ACS.

Definition A.2. Let Π be an n-party protocol where all parties take as input a global ACS property Q and each party P_i outputs a set S_i of parties. We say that Π is a t-resilient ACS protocol for Q if the following holds whenever up to t parties are corrupted:

- Consistency: Each honest party outputs the same set $S_i = S$.
- Set quality: Each output set has size at least $n - t$, and for each $P_i \in S$ there exists at least one honest party P_j that likes P_i for Q.
- Termination: All honest parties eventually terminate.

We also describe a functionality for ACS. In the functionality, each party can input $k \in [n]$. And it is guaranteed that every party receives at least $n - t$ such indices. Moreover, any index k input by a party P_i will also be eventually input by P_j.

For an ACS property Q, we will say that the parties invoke $\mathcal{F}_{\text{acs}}$, meaning that each party P_i inputs k to the functionality as soon as P_i likes P_k.

Functionality $\mathcal{F}_{\text{acs}}$

$\mathcal{F}_{\text{acs}}$ proceeds as follows, running with parties $P_1, \ldots, P_n$ and the adversary $\mathcal{S}$. Initialize $S_i = \varnothing$ for every $i \in [n]$, and $S = \perp$.

1: Upon receiving an index k from P_i, add index k to S_i. Then forward k to $\mathcal{S}$. If $|S_i| \geq n - t$, then we say that P_i is ready. If $n - t$ honest parties are ready, set S to be the set of indices k such that there is some honest party that inputs k.
2: Upon receiving S' from $\mathcal{S}$, check that $|S'| \geq n - t$, and that for every $k \in S'$, there is some honest party that has input k. If so, then set $S = S'$.

3: Upon setting S, output it to all parties as a request-based delayed output.

A.2 Preparing Random Coin

The description of $\mathcal{F}_{\text{Coin-Ab}}$ appears below. We mainly use it in the verification process introduced in Appendix E. The functionality can be realized by letting all parties first execute the generation phase of $\Pi_{\text{RandSh-Ab}}$ to prepare a batch of random degree-t Shamir sharing, then publicly reconstruct the random secret as the coin.

To generate random coins, all parties can execute the public reconstruction phase of $\Pi_{\text{RandSh-Ab}}$, but it requires $O(n^3)$ field elements communication complexity to generate one random coin (only the amortized cost is $O(n^2)$ when coins can be generated in parallel, while when we use $\mathcal{F}_{\text{Coin-Ab}}$ in our construction, all parties need to generate coins one by one.) To reduce it to $O(n^2)$, we change the public reconstruction process as follows:

(1) All parties execute $\Pi_{\text{RandSh-Ab}}$ to generate a batch of random degree-t Shamir sharing. For each generated random sharing $[r]_t$, if a party can compute his share of $[r]_t$, he sends it to all parties. Otherwise, he sends $\perp$ to all parties.
(2) Each party keeps waiting for messages from all parties. If he first uses OEC to reconstruct the secret r, he terminates with r. Otherwise, if he first receives $\perp$ from a party, he terminates with `Fail`.

The above construction also realizes $\mathcal{F}_{\text{Coin-Ab}}$, and the difference is that maybe some parties will terminate with r while others will terminate with `Fail`. So, the reconstruction of coins is not guaranteed to all parties compared to the previous construction. While this is sufficient for our construction when we use $\mathcal{F}_{\text{Coin-Ab}}$ in a black-box way. For the communication complexity, it still $O(n^3)$ field elements for preparing $O(n)$ random coins, but all parties can open them one by one (the previous construction require all parties open them simultaneously), and then the amortized cost per coin is $O(n^2)$ field elements.

Functionality $\mathcal{F}_{\text{Coin-Ab}}$

$\mathcal{F}_{\text{Coin}}$ proceeds as follows, running with parties $\mathcal{P} = \{P_1, \ldots, P_n\}$ and an adversary $\mathcal{S}$.

1: Upon receiving $2t+1$ parties' requests, the trusted party samples a random value r.
2: The trusted party sends a request-based delayed output r to all parties.
- Upon receiving $(\texttt{Fail}, P_i)$ from $\mathcal{S}$, if the output of P_i has not been delivered, change his output to `Fail`. Otherwise, ignore this request.

A.3 Distributed Zero Knowledge Proof

The functionality of $\mathcal{F}_{\text{dZK}}$ is defined below. To be self-contained, we briefly recall the construction of the distributed zero-knowledge (DZK) protocol from [ABCP23]. In this protocol, a dealer D aims to convince n verifiers $P_1, \ldots, P_n$ that a set of values $\{f_i(\alpha_\ell)\}_{i=1}^L$ held by each verifier P_ℓ are evaluations of valid degree-t polynomials. The dealer, acting as the prover, first samples a random degree-t *challenge* polynomial $Y(x)$ and computes its evaluations $Y(\alpha_1), \ldots, Y(\alpha_n)$. For each $\ell \in [n]$, the prover computes a commitment $C[\ell] := H(Y(\alpha_\ell))$ using a random oracle H, and broadcasts the vector $C = (C[1], \ldots, C[n])$. Then, the verifiers jointly sample a random challenge point $p \in \mathbb{F}$ and ask the prover to broadcast the polynomial $r(x) := Y(x) - \sum_{i=1}^L p^i f_i(x)$. Each verifier P_ℓ checks the consistency of the response by verifying whether

$$H\left(r(\alpha_\ell) + \sum_{i=1}^{L} p^i f_i(\alpha_\ell)\right) \stackrel{?}{=} C[\ell].$$

If the shares $\{f_i(x)\}$ are not valid degree-t polynomials, then $r(x)$ will not be degree-t, and the check fails with high probability due to the Schwartz-Zippel lemma, which bounds the probability that a non-zero polynomial evaluates to zero at a random point by at most $L/|\mathbb{F}|$. This protocol can be made non-interactive via the Fiat-Shamir heuristic by letting the prover compute the challenge point $p := H(C)$, thereby eliminating interaction.

Functionality $\mathcal{F}_{\text{dZK}}$

Public Input: $(\alpha_0, \ldots, \alpha_n), N$
$\mathcal{F}_{\text{dZK}}$ runs with parties $\mathcal{P} = \{P_1, \ldots, P_n\}$, a prover $P \in \mathcal{P}$, and an adversary $\mathcal{S}$.

1: Upon receiving polynomials $f_1(x), \ldots, f_N(x)$ from prover P, record them and send a requested-based delayed message $(\texttt{Delivered}, \texttt{DZK})$ to all parties.
2: Upon receiving $(\texttt{VerifyDZK}, s_1, \ldots, s_N, \alpha_k)$ from a party, if it has sent `Delivered`, $k \in [n]$ and $f_1(x), \ldots, f_N(x)$ are of degree-t and $s_1, \ldots, s_N$ equal $f_1(\alpha_k), \ldots, f_N(\alpha_k)$, send a requested-based delayed output `true` to this party. Otherwise, send a request-based delayed output `false` to this party.

A.4 Shamir Secret Sharing Scheme

In this work, we will use the standard Shamir Secret Sharing Scheme [Sha79]. Let n be the number of parties and $\mathbb{F}$ be a finite field of size $|\mathbb{F}| \geq 2n$. Let $\alpha_1, \ldots, \alpha_n$ be n distinct non-zero elements in $\mathbb{F}$.

A *degree-d* Shamir sharing of $x \in \mathbb{F}$ is a vector $(x_1, \ldots, x_n)$ which satisfies that there exists a polynomial $f(\cdot) \in \mathbb{F}[X]$ of degree at most d such that $f(0) = x$ and $f(\alpha_i) = x_i$ for $i \in \{1, \ldots, n\}$. Each party P_i holds a share x_i and the whole sharing is denoted by $[x]_d$. We recall the properties of the Shamir secret sharing scheme:

- Linear Homomorphism:

$$\forall\ [x]_d, [y]_d,\ [x+y]_d = [x]_d + [y]_d.$$

- Multiplying two degree-d sharings yields a degree-$2d$ sharing. The secret value of the new sharing is the product of the original two secrets.

$$\forall\ [x]_d, [y]_d,\ [x \cdot y]_{2d} = [x]_d \cdot [y]_d.$$

Reconstructing a degree-d Shamir sharing requires $d+1$ shares and can be done by Lagrange interpolation. For a random degree-d Shamir sharing of x, any d shares are independent of the secret x. If $d \geq t$, then knowing t of the shares does not leak anything about the k secrets. In particular, a sharing of degree t keeps hidden the underlying secret.

B ADDITIONAL OVERVIEWS

B.1 Differential Attack For Inconsistent Public Reconstruction

Previous work [GIP+14] has shown that when $n = 2t + 1$ and the network is synchronous, it is secure to let all parties exchange their shares of $[e]_{2t}$ to reconstruct e. In the asynchronous setting, since each party can expect to receive $n-t = 2t+1$ shares of $[e]_{2t}$, they can still reconstruct e. However, it is not secure because the adversary can do the following differential attack.

Assuming that all parties will compute $(x \cdot y) \cdot z$, where x, y, and z are some intermediate wire values in the circuit, and $([r]_t, [o]_{2t})$, $([r']_t, [o']_{2t})$ are two pairs of random *double sharings*. In the following, we denote the ith value of a Shamir secret sharing $[x]_d$ as x_i and assume that there are exactly t corrupted parties for simplicity.

(1) First, all parties compute gate $w = x \cdot y$, following the DN protocol, they will compute their shares of $[e]_{2t} := [x]_t \cdot [y]_t + [r]_t + [o]_{2t}$. All parties exchange their shares, and the adversary will do the following:
   - Let $\mathcal{H}'$ be the set of the first t honest parties, then sample a degree-$2t$ sharing of 1, denoted by $[I]_{2t}$, such that $I_i = 0$ for the $t + 1$ honest $P_i \notin \mathcal{H}'$.
   - Control corrupted party P_i to send $e_i + I_i$ to parties in $\mathcal{H}'$ and e_i to parties not in $\mathcal{H}'$.
   - Control the network delay to guarantee that each honest party will receive shares from the $t + 1$ honest parties not in $\mathcal{H}'$ and t corrupted parties.

   Therefore, parties not in $\mathcal{H}'$ will reconstruct the correct e and compute their shares of $[w]_t = e - [r]_t$. However, parties in $\mathcal{H}'$ will reconstruct the secret of $[e]_{2t} + [I]_{2t}$, which is $e + 1$. Then they will compute their shares of $[w]_t + 1$. The adversary follows the protocol to compute corrupted parties' shares of $[w]_t$.

(2) Then, all parties continue to compute $w \cdot z$. Parties in $\mathcal{H}'$ will compute shares of $[e']_{2t} + [z]_t := ([w]_t + 1) \cdot [z]_t + [r']_t + [o']_{2t}$, and honest parties not in $\mathcal{H}'$ will compute shares of $[e']_{2t} := [w]_t \cdot [z]_t + [r']_t + [o']_{2t}$. All parties exchange their shares, the adversary does the following to extract the secret z:
   - Compute t corrupted parties' shares of $[e']_{2t}$, then along with $t + 1$ shares of $[e']_{2t}$ received from parties not in $\mathcal{H}'$, recover the shares of $[e']_{2t}$ for parties in $\mathcal{H}'$.
   - Receive shares of $[e']_{2t} + [z]_t$ from t parties in $\mathcal{H}'$, then compute the difference with $[e']_{2t}$ and learn t shares of $[z]_t$.
   - Along with corrupted parties' shares of $[z]_t$ and t honest parties shares of $[z]_t$, the adversary recover the secret z.

The security problem comes from honest parties not using the same public reconstruction result. Note that this problem does not appear when $n = 2t + 1$ and the network is synchronous due to a smaller number of honest parties.

B.2 Reducing the Online Round Complexity

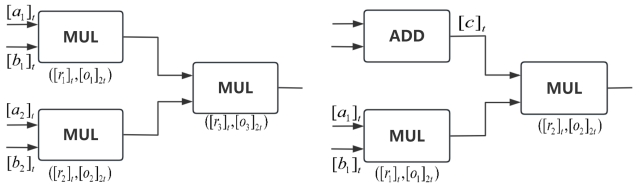
To achieve this goal, we first recall the technique of Beaver triple [Bea92]. Let $([a]_t, [b]_t, [c]_t)$ be a Beaver triple where $c = a \cdot b$, and $([x]_t, [y]_t)$ be the input sharings of a target multiplication gate, all parties do the following to compute the output wire $[z]_t$.

(1) Locally compute $[x+a]_t = [x]_t + [a]_t, [y+b]_t = [y]_t + [b]_t$.
(2) Publicly reconstruct $x + a, y + b$.
(3) Locally compute $[z]_t = (x + a)(y + b) - (x + a)[b]_t - (y + b)[a]_t + [c]_t$.

Based on the original Beaver triple technique, the observation in [GLO+21] is that only step 2 requires interaction, and if the input sharings are in the form of $(u, [a]_t), (v, [b]_t)$ where $u = x + a, v = y + b$, then all parties can locally compute $[z]_t = u \cdot v - u \cdot [b]_t - v \cdot [a]_t + [c]_t$ without any interaction. Therefore, if we can transform the input sharings from $[x]_t$ (resp. $[y]_t$) to $(u, [a]_t)$ (resp. $(v, [b]_t)$) before the evaluation of this gate, all parties only need to do local computation for this gate. The $(u, [a]_t)$ where $[x]_t = u - [a]_t$ is named *Beaver-triple friendly form* in [GLO+21].

To achieve this goal, we consider a two-layer circuit and will transform the output sharing for the first layer into *Beaver-triple friendly form*, then for the multiplication gates in the second layer, all parties only perform local computation. In figure 6, we show two cases and explain how it works. We will assign a fresh *double sharing* for each multiplication gate.

- For multiplication gates in the first layer, all parties use the DN technique to compute the output sharing.
- For case 1, recall that all parties will reconstruct $e_1 = a_1 \cdot b_1 + r_1$, $e_2 = a_2 \cdot b_2 + r_2$ during the DN protocol, then $(e_1, [r_1]_t), (e_2, [r_2]_t)$ are in the form of *Beaver-triple friendly form*. The only thing needed to do is compute $[r_1 \cdot r_2]_t$, which can be computed before all parties get $(e_1, [r_1]_t), (e_2, [r_2]_t)$. Therefore, they will also use the DN protocol to compute $[r_1 \cdot r_2]_t$, and the original *double sharing* which is assigned to the multiplication gate in the second layer will be consumed for the computation of $[r_1 \cdot r_2]_t$.
- For case 2, all parties will use $(e_1, [r_1]_t)$ and $(0, -[c]_t)$ as the inputs for the multiplication gate in the second layer, and they will compute $[r_1 \cdot c]_t$ in parallel.



(a) Case 1: A two-layer circuit with three MUL

(b) Case 2: A two-layer circuit with two MUL and one ADD

Figure 6: Two Cases for a two-layer circuit with three gates. Here $[o]_{2t}$ is a degree-$2t$ Shamir sharing of 0 and we use it to replace $[r]_{2t}$ in the *double sharing*, this is the observation in [GLO+21] which helps to decouple the relation between degree-t and $2t$ sharings.

To summarize, based on this idea, all parties can divide the whole circuit into many sub-two-layer circuits and evaluate each of them in parallel. The detailed construction is as follows, which is built based on our efficient multiplication protocol $\Pi_{\texttt{Mult}}$ introduced in section 7.

Protocol Π_{Eval}

All parties take a two-layer circuit as inputs and prepare random degree-t and $2t$ Shamir sharings for each multiplication gate.

1: All parties start with holding a degree-t Shamir sharing for each input wire of this circuit. For each multiplication gate in the second layer, we do the following to transform the input sharing to Beaver-triple friendly form $u - [a]_t$:
   - If it is an input sharing of the circuit, denoted by $[x]_t$, set $u := 0$ and $[a]_t := -[x]_t$.
   - If it is an output sharing of an addition gate in the first layer, first locally compute this sharing, denoted by $[x]_t$, and then set $u := 0$ and $[a]_t := -[x]_t$.
   - If it is an output sharing of a multiplication gate in the first layer, suppose $([r]_t, [o]_{2t})$ are associated with this gate. Set $[a]_t := [r]_t$. The value u will be determined later.
2: All parties do the following in parallel:
   - For each multiplication gate with input sharing $[x]_t, [y]_t$ in the first layer, all parties invoke $\Pi_{\mathtt{Mult}}$ to compute $[z]_t$ where $z := x \cdot y$. We denote the public reconstruction result in $\Pi_{\mathtt{Mult}}$ as e.
   - For each multiplication gate with input sharing $(u - [a]_t), (v - [b]_t)$ in the second layer, all parties invoke $\Pi_{\mathtt{Mult}}$ to compute $[c]_t$ where $c := a \cdot b$.
3: For each multiplication gate in the second layer, if its input sharing comes from the output sharing of a multiplication gate in the first layer, we set $u := e$. Then all parties have computed their input sharing $(u - [a]_t), (v - [b]_t)$ for each gate in the second layer and can locally compute:

$$[z]_t := u \cdot v - u \cdot [b]_t - v \cdot [a]_t + [c]_t$$

as the output sharing of this gate.

C SECURITY AND COST ANALYSIS OF PRIVATE SEND

C.1 Cost Analysis

We analyze the communication and round complexity as follows.

Communication Complexity. In the initialization phase, the dealer needs to send messages of size $O(1)$ to all parties and broadcast messages of size $O(1)$, resulting in $O(n^2)$ communication complexity. Then all parties help R to recover the symmetric key and invoke an instance of $\mathcal{F}_{\mathsf{ra}}$ to agree on the termination of the distributing phase, requiring $O(n)$ communication complexity in total.

For the same dealer, the initialization phase can be executed for distinct receivers in parallel, then the communication complexity of reliable broadcast is amortized to $O(n)$ to each receiver. Then the total communication complexity remains $O(n^2)$ field elements for n instances of $\Pi_{\mathtt{PrepKey}}$ when the dealer is the same.

In the disperse phase, let $|m|$ be the size of messages, the dealer needs to send messages of size $O(|m|/(t+1)+1)$ to all parties and broadcast a set $\mathcal{W}$ of size $O(n)$, and each party needs to broadcast messages of size $O(1)$ and forwards the messages received from dealer to R, result in $O(|m|+n^2)$ communication complexity in total, with the $|m|$ term specifically having a multiplicative constant of 5.

Round Complexity. The expected round complexity of reliable broadcast in [DXR21] is 4, and the reliable agreement in [DDL+24] is 2. Then in the initialization phase, it includes 6 rounds in the distributing phase and 1 round in the reconstruction phase, resulting in 7 rounds in total.

In the disperse phase, the broadcast executed by all parties can be parallel, then the round complexity is $4 + 1 + 4 + 1 = 10$.

C.2 Security Analysis

We prove lemma 4.1 and start with constructing the ideal adversary $\mathcal{S}$ as follows. Note that when the sender is corrupted, $\mathcal{S}$ can honestly follow the protocol to do a simulation. Therefore, we focus on the case when the sender is honest.

Simulator $\mathcal{S}$

When the sender is honest

Initialize Phase

1: $\mathcal{S}$ randomly samples values as corrupted parties' shares of $f(\alpha_j), g(\alpha_j)$ and sends them to corrupted parties on behalf of the dealer.
2: $\mathcal{S}$ randomly samples a value as c and reliably broadcasts it on behalf of the dealer.
3: For receiver R:
   - If R is corrupted, $\mathcal{S}$ randomly samples the whole sharing $[f(\alpha_0)]_t, [g(\alpha_0)]_t$ based on shares of corrupted parties, then sends each $f(\alpha_j), g(\alpha_j)$ to R on behalf of each honest party P_j. Then $\mathcal{S}$ maps $H(\mathsf{key}, r)$ to c. If $H(\mathsf{key}, r)$ has been mapped to other outputs before, $\mathcal{S}$ aborts the simulation. If c has been mapped to other inputs before, $\mathcal{S}$ also aborts the simulation.
   - If R is honest, when R receives shares from $t+1$ distinct parties, if any of them come from corrupted parties, $\mathcal{S}$ checks whether it equals the shares sampled by $\mathcal{S}$ before. If true, $\mathcal{S}$ delivers the output from $\mathcal{F}_{\mathsf{PrivSend\text{-}Ab}}$ to R. Otherwise, $\mathcal{S}$ sends $\mathtt{abort}$ to $\mathcal{F}_{\mathsf{PrivSend\text{-}Ab}}$.

Disperse Phase

4: $\mathcal{S}$ randomly samples values as each honest party P_i's h_i and broadcasts them.
5: $\mathcal{S}$ randomly samples values as each corrupted party P_i's $f(\alpha_i), g(\alpha_i)$, then $\mathcal{S}$ sends them to corrupted parties on behalf of the dealer. $\mathcal{S}$ also randomly samples values as c and broadcasts it on behalf of the dealer.
6: For receiver R, $\mathcal{S}$ does the same thing as the Initialize Phase to do simulation.

The hybrid arguments are as follows.

$\mathbf{Hyb}_0$: In this hybrid, we consider the execution in the real world.

$\mathbf{Hyb}_1$: In this hybrid, in the initialization phase, $\mathcal{S}$ delays the generation of honest parties' shares of $[f(\alpha_0)]_t, [g(\alpha_0)]_t$ until the reconstruction phase. Then for the generation of $c := H(\mathsf{key}, r)$, $\mathcal{S}$ directly randomly samples a value as c. Since honest parties' shares have not been used so far, the distributions of $\mathbf{Hyb}_1$ and $\mathbf{Hyb}_0$ are identical.

$\mathbf{Hyb}_2$: In this hybrid, in the initialization phase, for corrupted receiver R, $\mathcal{S}$ directly randomly samples the whole sharing $[f(\alpha_0)]_t, [g(\alpha_0)]_t$ based on shares of corrupted parties. Let $\mathsf{key} = f(\alpha_0), r = g(\alpha_0)$, if $H(\mathsf{key}, r)$ has been mapped to other outputs before, $\mathcal{S}$ aborts the simulation. If c has been mapped to other inputs before, $\mathcal{S}$ also aborts the simulation. If $|F| \geq 2^\kappa$, the probability is negligible in the security parameter κ. Therefore, the distributions of $\mathbf{Hyb}_2$ and $\mathbf{Hyb}_1$ are computationally indistinguishable.

$\mathbf{Hyb}_3$: In this hybrid, in the initialization phase, for honest receiver R, $\mathcal{S}$ also no longer requires honest dealer's secrets key, r. In the reconstruction phase, when R receives shares from corrupted parties, $\mathcal{S}$ sets R's output as `abort` if the shares received from corrupted parties are not equal to the shares sampled by $\mathcal{S}$ before. Therefore, the distributions of $\mathbf{Hyb}_3$ and $\mathbf{Hyb}_2$ are computationally indistinguishable.

$\mathbf{Hyb}_4$: In this hybrid, in the disperse phase, $\mathcal{S}$ change the generation of $H(\mathsf{key}, \mathtt{Id})$, he will first randomly sample a value as $c_{\mathtt{Id}}$ and then computes $H(\mathsf{key}, \mathtt{Id}) = c_{\mathtt{Id}} \oplus m_{\mathtt{Id}}$. Since both $c_{\mathtt{Id}}$ and $H(\mathsf{key}, \mathtt{Id})$ are randomly sampled, this makes no difference. Therefore, the distributions of $\mathbf{Hyb}_4$ and $\mathbf{Hyb}_3$ are identical.

$\mathbf{Hyb}_5$: In this hybrid, in the disperse phase, $\mathcal{S}$ does the same thing as in $\mathbf{Hyb}_1, \mathbf{Hyb}_2$ and $\mathbf{Hyb}_3$. The distributions of $\mathbf{Hyb}_5$ and $\mathbf{Hyb}_4$ are computationally indistinguishable.

Note that $\mathbf{Hyb}_5$ corresponds to the ideal world, then $\Pi_{\mathtt{PrivSend\text{-}Ab}}$ securely computes $\mathcal{F}_{\mathtt{PrivSend\text{-}Ab}}$ when the dealer is honest.

D SECURE AND COST ANALYSIS OF GENERATING RANDOM SHARINGS

D.1 Analysis for $\Pi_{\mathtt{ACSS\text{-}Ab}}$

We analyze the communication complexity, round complexity, and security as follows.

Communication Complexity. In the distributing phase, the dealer needs to send $2t \cdot L = \frac{2}{3}nL$ messages to the last $2t+1$ parties. In the verification phase, each party will broadcast $H(r_i)$, which results in $O(n^3)$ additive overhead. Then each of them will send a confirming message to the dealer, and the dealer will reliably broadcast `Fail` or a set $\mathcal{M}$ of size $2t+1$. This will cause $O(n^2)$ additive overhead in total. If the dealer broadcasts $\mathcal{M}$, then he needs to invoke an instance of $\mathcal{F}_{\mathtt{dZK}}$ and t instances of $\mathcal{F}_{\mathtt{PrivSend\text{-}Ab}}$. The instance of $\mathcal{F}_{\mathtt{dZK}}$ requires $O(n^2)$ additive overhead, and each instance of $\mathcal{F}_{\mathtt{PrivSend\text{-}Ab}}$ requires $5L$ field elements communication and $O(n)$ additive overhead. Note that there is no interaction in the termination phase, so the total communication complexity in the sharing phase is $O(Ln + n^3)$ field elements, with the L term specifically having a multiplicative constant of $\frac{5}{3} + \frac{2}{3} = \frac{7}{3}$.

Note that the $O(n^3)$ additive overhead is only caused by all parties broadcasting $H(r_i)$ in the confirming phase. This step can be amortized to $O(n^2)$ if all parties participate in $\Pi_{\mathtt{ACSS\text{-}Ab}}$ invoked by n dealers in parallel. So in the later construction of $\Pi_{\mathtt{RandSh\text{-}Ab}}$, all parties will invoke n instances of $\Pi_{\mathtt{ACSS\text{-}Ab}}$ led by different dealers, and the additive overhead is still $O(n^3)$.

In the public reconstruction phase, the communication complexity is $O(Ln^2)$ field elements.

Round Complexity. In the distributing phase, it requires 1 round for the dealer to distribute messages and an instance of $\mathcal{F}_{\mathtt{dZK}}$. According to the construction in [ABCP23], it requires 4 rounds. Then in the verification phase, all parties require 4 rounds to broadcast $H(r_i)$, this step can be executed with the distributing phase in parallel. Then, all parties need to send a confirmation to the dealer, which requires 1 round. The dealer then broadcasts a message that requires 4 rounds. If the messages is a set $\mathcal{M}$, the dealer needs to further invokes $\mathcal{F}_{\mathtt{PrivSend\text{-}Ab}}$:

- For the instance of $\mathcal{F}_{\mathtt{PrivSend\text{-}Ab}}$, all parties will participate in when they receive the set $\mathcal{M}$ from D, it requires 10 rounds according to our previous analysis in appendix C.1. Note that among these 10 rounds, 4 of them also come from the broadcast in the first step of $\Pi_{\mathtt{AVID\text{-}Ab}}$, and we can move this step to the beginning of $\Pi_{\mathtt{ACSS\text{-}Ab}}$. Then it only requires 6 rounds for the instance of $\mathcal{F}_{\mathtt{PrivSend\text{-}Ab}}$.

Therefore, the total round complexity in the sharing phase is computed as follows:

$$\max(1, 4) + 1 + 4 + 6 = 15$$

In the public reconstruction phase, it requires 1 round for all parties to reconstruct the output.

Security Analysis. We prove lemma 5.1 and start with constructing the ideal adversary $\mathcal{S}$ as follows. We first consider the case of an honest dealer.

Simulator $\mathcal{S}$

When the dealer is honest

Sharing Phase

1: $\mathcal{S}$ invokes $\mathcal{F}_{\mathtt{ACSS\text{-}Ab}}$ and receives shares of corrupted parties. Then $\mathcal{S}$ sends them to each corrupted party P_i on behalf of D.
2: For each honest party P_i, $\mathcal{S}$ follows the protocol to broadcast a value as $H(r_i)$. Then $\mathcal{S}$ honestly follows the protocol to learn the message which will be broadcast by the dealer, and broadcasts it on behalf of D. If the message is the set $\mathcal{M}$, $\mathcal{S}$ honestly simulates each $\mathcal{F}^{(i)}_{\mathtt{PrivSend\text{-}Ab}}$ for $P_i \notin \mathcal{M}$ and simulates $\mathcal{F}_{\mathtt{dZK}}$ as follows.
(1). When D's inputs has been delivered to $\mathcal{F}_{\mathtt{dZK}}$, $\mathcal{S}$ sends $(\mathtt{Delivered}, \mathtt{ACSS})$ to corrupted parties on behalf of $\mathcal{F}_{\mathtt{dZK}}$.
(2). When $\mathcal{S}$ receives request $(\mathtt{VerifyDZK}, s_1, \ldots, s_N, \alpha_k)$ from a corrupted party, if k is an index of a corrupted party and $s_1, \ldots, s_N$ equal $f_1(\alpha_k), \ldots, f_L(\alpha_k)$, $\mathcal{S}$ sends `true` to this corrupted party as the output of $\mathcal{F}_{\mathtt{dZK}}$. Otherwise, $\mathcal{S}$ sends `false` to this corrupted party as the output of $\mathcal{F}_{\mathtt{dZK}}$.
If the message is `abort`, $\mathcal{S}$ sends `Fail` to $\mathcal{F}_{\mathtt{ACSS\text{-}Ab}}$.
3: For each honest party P_i whose output is `abort`, $\mathcal{S}$ sends $(\mathtt{abort}, P_i)$ to $\mathcal{F}_{\mathtt{ACSS\text{-}Ab}}$.

Reconstruction Phase

4: $\mathcal{S}$ learns $q_1(x), \ldots, q_L(x)$ from $\mathcal{F}_{\mathtt{ACSS\text{-}Ab}}$, then honestly simulates the protocol on behalf of each honest party.
5: $\mathcal{S}$ outputs the views of $\mathcal{A}$.

The hybrid arguments are as follows.

$\mathbf{Hyb}_0$: In this hybrid, we consider the execution in the real world.

$\mathbf{Hyb}_1$: In this hybrid, $\mathcal{S}$ first generate corrupted parties' shares. Then he samples honest parties' shares based on the secret $\{f_1(\alpha_\ell)\}_{\ell \in [L]}$ and shares of corrupted parties. Based on the property of the Shamir sharing scheme, this makes no difference. The distributions of $\mathbf{Hyb}_1$ and $\mathbf{Hyb}_0$ are identical.

$\mathbf{Hyb}_2$: In this hybrid, In this hybrid, $\mathcal{S}$ change the simulation of each $\mathcal{F}_{\mathtt{dZK}}$ as above. The difference between $\mathbf{Hyb}_1$ and $\mathbf{Hyb}_0$ is $\mathcal{S}$ will reply `false` when the adversary sends request

$$(\mathtt{VerifyDZK}, \{f_\ell(\alpha_i)\}_{\ell \in [L']}, \alpha_i)$$

to $\mathcal{F}_{\mathtt{dZK}}$ and i is the index of an honest party. That means the adversary can correctly guess the shares of honest parties, which is negligible in the security parameter. Therefore, the distributions of $\mathbf{Hyb}_2$ and $\mathbf{Hyb}_1$ are statistically close.

$\mathbf{Hyb}_3$: In this hybrid, $\mathcal{S}$ no longer generates honest parties' shares since they are never used. Then $\mathcal{S}$ also does not require the dealer's secrets $f_1(\alpha_0), \ldots, f_L(\alpha_0)$ in the sharing phase. He will learn them from $\mathcal{F}_{\text{ACSS-Ab}}$ during the public reconstruction phase. The distributions of $\mathbf{Hyb}_3$ and $\mathbf{Hyb}_2$ are identical.

Note that $\mathbf{Hyb}_3$ corresponds to the ideal world, then $\Pi_{\text{ACSS-Ab}}$ securely computes $\mathcal{F}_{\text{ACSS-Ab}}$ when the dealer is honest.

Then we consider the case of a corrupted dealer.

Simulator $\mathcal{S}$

When the dealer is corrupted

Sharing Phase

1: $\mathcal{S}$ honestly executes each honest party and learns their outputs. For each honest party whose output is `abort`, $\mathcal{S}$ sends $(\texttt{abort}, P_i)$ to $\mathcal{F}_{\text{ACSS-Ab}}$.
2: If $\mathcal{S}$ receives `Fail` from D, $\mathcal{S}$ sends `Fail` to $\mathcal{F}_{\text{ACSS-Ab}}$. If $\mathcal{S}$ receives a valid set $\mathcal{M}$ from D, let $\mathcal{H}$ denote the set of the first $t+1$ honest parties who have accepted their shares. Then $\mathcal{S}$ interpolates polynomials $f_1(x), \ldots, f_L(x)$ and sends them to $\mathcal{F}_{\text{ACSS-Ab}}$ and deliveries the output from $\mathcal{F}_{\text{ACSS-Ab}}$ to all honest parties.

Reconstruction Phase

3: $\mathcal{S}$ honestly follows the protocol.
4: $\mathcal{S}$ outputs the views of $\mathcal{A}$.

The hybrid arguments are as follows.

$\mathbf{Hyb}_0$: In this hybrid, we consider the execution in the real world.

$\mathbf{Hyb}_1$: In this hybrid, $\mathcal{S}$ uses the first $t+1$ honest parties' shares to reconstruct degree-t polynomials $f_1(x), \ldots, f_L(x)$. For an honest party who accepts his shares but does not lie on degree-t polynomials $f_1(x), \ldots, f_L(x)$, $\mathcal{S}$ aborts the simulation. Since DZK guarantees that all parties only accept their shares if their shares are consistent, $\mathcal{S}$ will never abort. The distributions between $\mathbf{Hyb}_1$ and $\mathbf{Hyb}_0$ are identical.

$\mathbf{Hyb}_2$: In this hybrid, $\mathcal{S}$ sends $f_1(x), \ldots, f_L(x)$ to $\mathcal{F}_{\text{ACSS-Ab}}$ and let it deliver the output to each party. The distributions between $\mathbf{Hyb}_2$ and $\mathbf{Hyb}_1$ are identical.

Note that $\mathbf{Hyb}_2$ corresponds to the ideal world, then $\Pi_{\text{ACSS-Ab}}$ securely computes $\mathcal{F}_{\text{ACSS-Ab}}$ when the dealer is corrupted.

D.2 Analysis for $\Pi_{\text{Sh2t-Ab}}$

We analyze the communication complexity, round complexity, and security as follows.

Communication Complexity. In the distributing phase, the dealer directly invoke t instances of $\mathcal{F}_{\text{PrivSend-Ab}}$ for the last t parties, which requires $\frac{5}{3}Ln$ field elements communication and $O(n^2)$ field elements additive overhead.

Round Complexity. According to our previous analysis of $\mathcal{F}_{\text{PrivSend-Ab}}$ in appendix C.1, it requires 10 rounds in total.

Security Analysis. We prove lemma 5.2 and start with constructing the ideal adversary $\mathcal{S}$ as follows. When the dealer is honest, $\mathcal{S}$ can honestly follow the protocol to do the simulation, then we consider the case of a corrupted dealer.

Simulator $\mathcal{S}$

When the dealer is corrupted

1: $\mathcal{S}$ honestly executes each honest party and learns shares of honest parties. For each honest party P_i whose output is `abort`, he sends $(\texttt{abort}, P_i)$ to $\mathcal{F}_{\text{Sh2t-Ab}}$.
2: When honest parties receive $(\texttt{Delivered}, \texttt{Sh2t})$ from $\mathcal{F}^{(i)}_{\text{PrivSend-Ab}}$ for all $i \in [2t+1, n]$, $\mathcal{S}$ interpolates $f_1(x), \ldots, f_L(x)$ based on all parties shares. Note that $\mathcal{S}$ has learn each P_i's shares $f_\ell(\alpha_i)$ during the simulation of $\mathcal{F}^{(i)}_{\text{PrivSend-Ab}}$ or extracts it from $H(\text{key}_i)$.
3: $\mathcal{S}$ sends $f_1(x), \ldots, f_L(x)$ to $\mathcal{F}_{\text{Sh2t-Ab}}$ and deliveries the output from $\mathcal{F}_{\text{Sh2t-Ab}}$ to each party.
4: $\mathcal{S}$ outputs the views of $\mathcal{A}$.

The hybrid arguments are as follows.

$\mathbf{Hyb}_0$: In this hybrid, we consider the execution in the real world.

$\mathbf{Hyb}_1$: In this hybrid, $\mathcal{S}$ uses the $2t+1$ honest parties' shares to reconstruct degree-$2t$ polynomials $f_1(x), \ldots, f_L(x)$. Then he sends them to $\mathcal{F}_{\text{Sh2t-Ab}}$ and lets it deliver the output to each party. The distributions between $\mathbf{Hyb}_1$ and $\mathbf{Hyb}_0$ are identical.

Note that $\mathbf{Hyb}_1$ corresponds to the ideal world, then $\Pi_{\text{Sh2t-Ab}}$ securely computes $\mathcal{F}_{\text{Sh2t-Ab}}$ when the dealer is corrupted.

D.3 Analysis for $\Pi_{\text{RandSh-Ab}}$

Communication Complexity. It includes n instances of $\Pi_{\text{ACSS-Ab}}$ and an instances of $\mathcal{F}_{\text{acs}}$, the total communication complexity is $O(Nn + n^3)$ field elements, with the N term specifically having a multiplicative constant of 7.

Security Analysis. We prove lemma 6.1 and start with constructing the ideal adversary $\mathcal{S}$ as follows.

Simulator $\mathcal{S}$

1: $\mathcal{S}$ simulates $\mathcal{F}_{\text{ACSS-Ab}}$ led by P_i as follows:
- If P_i is honest, $\mathcal{S}$ randomly samples values as shares of corrupted parties. Then $\mathcal{S}$ sends them to corrupted parties.
- If P_i is corrupted, $\mathcal{S}$ learns P_i's inputs polynomials.

2: $\mathcal{S}$ honestly simulates $\mathcal{F}_{\text{acs}}$ and learns the set $\mathcal{D}$. For each honest party P_i whose output is `abort` or `Fail`, $\mathcal{S}$ sends $(\texttt{Fail}, P_i)$ to $\mathcal{F}_{\text{RandSh-Ab}}$.
3: $\mathcal{S}$ computes corrupted parties' shares of $\{[r_{\ell,i}]_t\}_{\ell \in [N'], i \in [t+1]}$ and sends them to $\mathcal{F}_{\text{RandSh-Ab}}$, when honest parties receives their output from each $\mathcal{F}_{\text{ACSS-Ab}}$ led by dealers in $\mathcal{D}$, $\mathcal{S}$ delivers the output from $\mathcal{F}_{\text{RandSh-Ab}}$ to all parties.

The hybrid arguments are as follows.

$\mathbf{Hyb}_0$: In this hybrid, we consider the execution in the real world.

$\mathbf{Hyb}_1$: In this hybrid, for honest dealers, $\mathcal{S}$ samples random values as shares of corrupted parties. Then he sends them to corrupted parties on behalf of $\mathcal{F}_{\text{ACSS-Ab}}$ invoked by honest dealers. $\mathcal{S}$ delays the generation of honest parties' shares until the set $\mathcal{D}$ is determined. The distributions between $\mathbf{Hyb}_1$ and $\mathbf{Hyb}_0$ are identical.

$\mathbf{Hyb}_2$: In this hybrid, $\mathcal{S}$ first compute corrupted parties' shares of $\{[r_{\ell,i}]_t\}_{\ell \in [N'], i \in [t+1]}$, then randomly samples the whole sharing $\{[r_{\ell,i}]_t\}_{\ell \in [N'], i \in [t+1]}$ based on shares of corrupted parties. Let $\mathcal{H}$ be the set of the first $t+1$ honest dealers in $\mathcal{D}$ and $Corr = \mathcal{D} \setminus \mathcal{H}$,

then:

$$([r_{\ell,i}]_t)_{i\in[t+1]} = \mathcal{M}_{\mathcal{H}} \cdot ([s_\ell^{(i)}]_t)_{i\in\mathcal{H}} + \mathcal{M}_{Corr} \cdot ([s_\ell^{(i)}]_t)_{i\in Corr}$$

Then $\mathcal{S}$ computes the whole sharings $([s_\ell^{(i)}]_t)_{i\in\mathcal{H}}$ based on based on $([r_{\ell,i}]_t)_{i\in[t+1]}$ and $([s_\ell^{(i)}]_t)_{i\in Corr}$. The distributions between $\mathbf{Hyb}_2$ and $\mathbf{Hyb}_1$ are identical since there exists a one-to-one map between $([r_{\ell,i}]_t)_{i\in[t+1]}$ and $([s_\ell^{(i)}]_t)_{i\in Corr}$.

$\mathbf{Hyb}_3$: In this hybrid, $\mathcal{S}$ no longer generates honest parties' shares since they are never used. He sends $\{[r_{\ell,i}]_t\}_{\ell\in[N'],i\in[t+1]}$ to $\mathcal{F}_{\mathsf{RandSh\text{-}Ab}}$ and let it distributes the outputs to all parties. The distributions between $\mathbf{Hyb}_3$ and $\mathbf{Hyb}_2$ are identical.

Note that $\mathbf{Hyb}_3$ corresponds to the ideal world, then $\Pi_{\mathsf{RandSh\text{-}Ab}}$ securely computes $\mathcal{F}_{\mathsf{RandSh\text{-}Ab}}$.

D.4 Analysis for $\Pi_{\mathsf{RandShZero\text{-}Ab}}$

Communication Complexity. It includes n instances of $\Pi_{\mathsf{Sh2t\text{-}Ab}}$ and an instances of $\mathcal{F}_{\mathsf{acs}}$, the total communication complexity is $O(Nn + n^3)$ field elements, with the N term specifically having a multiplicative constant of 5.

Security Analysis. We prove lemma 6.2 and start with constructing the ideal adversary $\mathcal{S}$ as follows.

Simulator $\mathcal{S}$

1: $\mathcal{S}$ simulates $\mathcal{F}_{\mathsf{Sh2t\text{-}Ab}}$ led by P_i as follows:
- If P_i is honest, $\mathcal{S}$ randomly samples values as shares of corrupted parties. Then $\mathcal{S}$ sends them to corrupted parties.
- If P_i is corrupted, $\mathcal{S}$ learns P_i's inputs polynomials.

2: $\mathcal{S}$ honestly simulates $\mathcal{F}_{\mathsf{acs}}$ and learns the set $\mathcal{D}$. For each honest party P_i whose output is $\mathtt{abort}$, $\mathcal{S}$ sends $(\mathtt{abort}, P_i)$ to $\mathcal{F}_{\mathsf{RandSh\text{-}Ab}}$.

3: For each corrupted dealer $P_i \in \mathcal{D}$, $\mathcal{S}$ receives his polynomial $q_1^{(i)}(x), \ldots, q_{N'}^{(i)}(x)$ from P_i. For each $\ell \in [N']$ and $P_i \in \mathcal{D}$, if P_i is corrupted, $\mathcal{S}$ computes a vector $\boldsymbol{q}_\ell^{(i)} = (q_\ell^{(i)}(\alpha_1), \ldots, q_\ell^{(i)}(\alpha_n))$. Otherwise, if P_i is honest, $\mathcal{S}$ sets $\boldsymbol{q}_\ell^{(i)} = (0, \cdots, 0)$. Then for all corrupted dealers in $\mathcal{D}$, $\mathcal{S}$ computes:

$$(\boldsymbol{e}_{\ell,1}, \ldots, \boldsymbol{e}_{\ell,t+1}) = \boldsymbol{M} \cdot (\boldsymbol{q}_\ell^{(i)})_{i\in\mathcal{D}}.$$

Then $\mathcal{S}$ transform $\{\boldsymbol{e}_{\ell,k}\}_{\ell\in[N'],k\in[t+1]}$ to $\{\boldsymbol{e}_\ell\}_{\ell\in[N]}$. For each $\ell \in [N]$, $\mathcal{S}$ sends $\boldsymbol{e}_\ell$ to $\mathcal{F}_{\mathsf{RandShZero\text{-}Ab}}$.

4: $\mathcal{S}$ outputs the views of $\mathcal{A}$.

The hybrid arguments are as follows.

$\mathbf{Hyb}_0$: In this hybrid, we consider the execution in the real world.

$\mathbf{Hyb}_1$: In this hybrid, for honest dealers, $\mathcal{S}$ samples random values as shares of corrupted parties. Then he sends them to corrupted parties on behalf of $\mathcal{F}_{\mathsf{Sh2t\text{-}Ab}}$ invoked by honest dealers. $\mathcal{S}$ delays the generation of honest parties' shares until the set $\mathcal{D}$ is determined. The distributions between $\mathbf{Hyb}_1$ and $\mathbf{Hyb}_0$ are identical.

$\mathbf{Hyb}_2$: In this hybrid, $\mathcal{S}$ first compute corrupted parties' shares of $\{[o_{\ell,i}]_t\}_{\ell\in[N'],i\in[t+1]}$, then randomly samples the whole sharing $\{[o_{\ell,i}]_t\}_{\ell\in[N'],i\in[t+1]}$ based on shares of corrupted parties. Let $\mathcal{H}$ be the set of the first $t+1$ honest dealers in $\mathcal{D}$ and $Corr = \mathcal{D} \setminus \mathcal{H}$, then:

$$([o_{\ell,i}]_t)_{i\in[t+1]} = \mathcal{M}_{\mathcal{H}} \cdot ([o_\ell^{(i)}]_t)_{i\in\mathcal{H}} + \mathcal{M}_{Corr} \cdot ([o_\ell^{(i)}]_t)_{i\in Corr}$$

Then $\mathcal{S}$ computes the whole sharings $([o_\ell^{(i)}]_t)_{i\in\mathcal{H}}$ based on based on $([o_{\ell,i}]_t)_{i\in[t+1]}$ and $([o_\ell^{(i)}]_t)_{i\in Corr}$. The distributions between $\mathbf{Hyb}_2$ and $\mathbf{Hyb}_1$ are identical since there exists a one-to-one map between $([o_{\ell,i}]_t)_{i\in[t+1]}$ and $([o_\ell^{(i)}]_t)_{i\in Corr}$.

$\mathbf{Hyb}_3$: In this hybrid, $\mathcal{S}$ no longer generates honest parties' shares since they are never used. He computes $\{\boldsymbol{e}_\ell\}_{\ell\in[N]}$ as above and sends $\{[o_{\ell,i}]_t\}_{\ell\in[N'],i\in[t+1]}$ to $\mathcal{F}_{\mathsf{RandShZero\text{-}Ab}}$ and let it distributes the outputs to all parties. The distributions between $\mathbf{Hyb}_3$ and $\mathbf{Hyb}_2$ are identical.

Note that $\mathbf{Hyb}_3$ corresponds to the ideal world, then $\Pi_{\mathsf{RandShZero\text{-}Ab}}$ securely computes $\mathcal{F}_{\mathsf{RandShZero\text{-}Ab}}$.

D.5 Generating Random Shamir Sharings in Pipeline

To reduce the constant factor in communication complexity, we can apply a pipelining technique parameterized by an input factor k. For sufficiently large k, this optimization reduces the communication constant by up to 1/3, at the cost of increased round complexity. In practice, k should be chosen to balance communication and round efficiency based on application requirements.

Protocol $\Pi_{\mathsf{RandShPipe}}$

Let N be the number of Shamir sharing needed to be generated, k be a constant number, and $\Pi \in \{\Pi_{\mathsf{RandSh\text{-}Ab}}, \Pi_{\mathsf{RandShZero\text{-}Ab}}\}$.

1: All parties divide the generation process into k segments, and assign an index $\ell \in [k]$ for each segment.

2: In each segment, all parties execute an instance of Π to generate N/k random sharings. We denote $\Pi^{(\ell)}$ as the instance of Π executed in the ℓth segment, $\mathcal{D}^{(\ell)}$ as the set of successful dealers in $\Pi^{(\ell)}$. All parties locally initializes a global flag $\mathsf{unused}_i = \mathtt{false}$ for each party P_i, then they do the following in each segment:
- If $\mathsf{unused}_i = \mathtt{false}$, all parties participate in $\mathcal{F}_{\mathsf{ACSS\text{-}Ab}}$ or $\mathcal{F}_{\mathsf{Sh2t\text{-}Ab}}$ invoked by the dealer P_i. When they terminate, they set $\mathsf{unused}_i = \mathtt{true}$.
- During the agreement of successful dealers, each party P_i sets the property Q as he has set $\mathsf{used}_j = \mathtt{true}$.
- Upon agree on the set $\mathcal{D}^{(\ell)}$, all parties set $\mathsf{unused}_i = \mathtt{false}$ for each $P_i \in \mathcal{D}^{(\ell)}$. Then they follow the protocol to extract random sharings, noting that the random sharings distributed by some $P_i \in \mathcal{D}^{(\ell)}$ may come from one of the previous segments.

E VERIFICATION OF MULTIPLICATION TUPLES

In this section, we introduce the verification process for multiplication tuples. The idea comes from [GS20] and we apply it to the asynchronous setting. The achieved communication complexity is $O(n^2 \log N + n^3)$ field elements for verifying N multiplication tuples. To be self-contained, we attach the construction as follows. We refer the readers who are interested in this part to check [GS20] for the high-level ideas.

Functionality of Multiplication Tuples Verify. The functionality is defined below.

Functionality $\mathcal{F}_{\mathsf{MulTupleVrfy}}$

$\mathcal{F}_{\mathsf{MulTupleVrfy}}$ runs with parties $\mathcal{P} = \{P_1, \ldots, P_n\}$ and an adversary $\mathcal{S}$.

1: Let N denote the number of multiplication tuples. The multiplication tuples to be verified are denoted by $\{([x_\ell]_t, [y_\ell]_t, [z_\ell]_t)\}_{\ell=1}^N$.
2: For all $\ell \in [N]$, upon receiving shares of $([x_\ell]_t, [y_\ell]_t, [z_\ell]_t)$ from $t+1$ distinct honest parties, the trusted party reconstructs the whole sharings $([x_\ell]_t, [y_\ell]_t, [z_\ell]_t)$ and proceeds. If it receives $\perp$ from at least $t+1$ distinct honest parties, it gives up security, and lets $\mathcal{S}$ determine all honest parties' outputs.
3: The trusted party computes $d_\ell = z_\ell - x_\ell \cdot y_\ell$ for all $\ell \in [N]$. Then it sends $\{d_\ell\}_{\ell \in [N]}$ and corrupted parties' shares of $([x_\ell]_t, [y_\ell]_t, [z_\ell]_t)$ to $\mathcal{S}$.
4: Let $b \in \{\mathtt{Fail}, \mathtt{Success}\}$ denote the verification result, then it sends a request-based delayed output b to all parties.
- Upon receiving a request `Fail` from $\mathcal{S}$, if the output of all parties has not been delivered, change the output of all parties by `Fail`. Otherwise, ignore this request.

Computing Vector of Multiplication Tuples. In the following, we use $\odot$ to denote the inner product. Π_{ExMult} allows all parties to compute the inner product of the vectors of multiplication tuples.

Protocol Π_{ExMult}

The protocol is the same as $\Pi_{\mathtt{Mult}}$, except that:

- All parties take their shares of $\{[\boldsymbol{a}^{(\ell)}]_t, [\boldsymbol{b}^{(\ell)}]_t\}_{\ell=1}^N$ as inputs, $\boldsymbol{a}^{(\ell)}, \boldsymbol{b}^{(\ell)}$ are vectors of Shamir sharings.
- In step 4 of $\Pi_{\mathtt{Mult}}$, set each $[z_k]_{2t} = [a_k]_t \odot [b_k]_t + [r_k]_t$.

Compress Multiplication Tuples. Π_{ExCompr} allows all parties to compress a batch of vectors of multiplication tuples into one vector of multiplication tuples.

Protocol Π_{ExCompr}

Let $\{[\boldsymbol{x}^{(i)}]_t, [\boldsymbol{y}^{(i)}]_t, [z^{(i)}]_t\}_{i=1}^N$ be the vector of tuples which needed to be compressed and $\alpha_1, \ldots, \alpha_{2N-1}$ be distinct field elements. All parties require $N-1$ fresh $[r]_t$ and $(N-1)/2$ fresh $[o]_{2t}$.

1: Let $\boldsymbol{f}(\cdot), \boldsymbol{g}(\cdot)$ be vector of degree-$(N-1)$ polynomials such that:
$$\forall i \in [N], \boldsymbol{f}(\alpha_i) = \boldsymbol{x}^{(i)}, \boldsymbol{g}(\alpha_i) = \boldsymbol{y}^{(i)}$$
Then all parties locally compute $[\boldsymbol{f}(\cdot)]_t, [\boldsymbol{g}(\cdot)]_t$ by using $\{[\boldsymbol{x}^{(i)}]_t\}_{i=1}^N$ and $\{[\boldsymbol{y}^{(i)}]_t\}_{i=1}^N$ respectively.
2: For all $i \in [N+1, 2N-1]$, all parties locally compute $[\boldsymbol{f}(\alpha_i)]_t, [\boldsymbol{g}(\alpha_i)]_t$, and then invoke Π_{ExMult} on $[\boldsymbol{f}(\alpha_i)]_t, [\boldsymbol{g}(\alpha_i)]_t$ to compute $[z^{(i)}]_t = [\boldsymbol{f}^{(i)} \odot \boldsymbol{g}^{(i)}]_t$.
3: Let $h(\cdot)$ be a degree-$2(N-1)$ polynomial such that:
$$\forall i \in [2N-1], h(\alpha_i) = z^{(i)}.$$
Then all parties locally compute $[h(\cdot)]_t$ by using $\{[z^{(i)}]_t\}_{i=1}^{2N-1}$.
4: All parties invoke $\mathcal{F}_{\mathsf{Coin\text{-}Ab}}$ to generate a random element r. For each party, if $r \in [\alpha_1, \ldots, \alpha_N]$ or he receives `Fail` from $\mathcal{F}_{\mathsf{Coin\text{-}Ab}}$, he outputs `Fail`. Otherwise, they output $\{[\boldsymbol{f}(r)]_t, [\boldsymbol{g}(r)]_t, [h(r)]_t\}$.

Verification of Multiplication Tuples. During the verification process, all parties need to agree on random common challenges. This is realized by the functionality $\mathcal{F}_{\mathsf{Coin\text{-}Ab}}$ defined in Appendix A.2. Note that $\mathcal{F}_{\mathsf{Coin\text{-}Ab}}$ is also a weaker version, which does not guarantee that all honest parties can get the challenge. It only ensures that the challenges accepted by all parties are the same. We use this weaker version to bound the overhead within amortized $O(n^2)$ for generating each random challenge in each iteration.

Protocol $\Pi_{\mathsf{MulTupleVrfy}}$

Let k be the compression parameter and $\{[a^{(\ell)}]_t, [b^{(\ell)}]_t, [c^{(\ell)}]_t\}_{\ell=1}^N$ be the multiplication tuples to be verified. In the following, if a party receives `Fail` from functionality or sub-protocols, he will set his output decision as `Fail`. If a party takes $\perp$ as input, he also sets his output decision as `Fail`.

1: **Step 1: Preparation of Double Sharings.**
Let $N' = 2(k-1)(\lfloor \log_k N \rfloor + 1)$, all parties invoke $\mathcal{F}_{\mathsf{RandSh\text{-}Ab}}$(resp. $\mathcal{F}_{\mathsf{RandShZero\text{-}Ab}}$) to prepare N'(resp. $N'/2$) degree-t(resp. $2t$) Shamir sharings, when they require fresh sharings in the sub-protocols, they pick the first unused one among them.
2: **Step 2: De-Linearization.**
(1). All parties invoke $\mathcal{F}_{\mathsf{Coin\text{-}Ab}}$ to get a random element r.
(2). All parties set:
$$[f]_t = ([a^{(1)}]_t, r[a^{(2)}]_t, \ldots, r^{N-1}[a^{(N)}]_t),$$
$$[\boldsymbol{g}]_t = ([b^{(1)}]_t, [b^{(2)}]_t, \ldots, [b^{(N)}]_t),$$
$$[h]_t = \sum_{\ell=1}^N r^{\ell-1}[c^{(\ell)}]_t$$
3: **Step 3: Dimension-Reduction**
All parties initialize $\{[\boldsymbol{x}]_t, [\boldsymbol{y}]_t, [z]_t\} = \{[f]_t, [\boldsymbol{g}]_t, [h]_t\}$, then they repeatedly do the following things until the dimension of $\{[\boldsymbol{x}]_t, [\boldsymbol{y}]_t, [z]_t\}$ is less than k. Let L denote the dimension of the inner-product tuple (i.e. the dimension of vector $\boldsymbol{x}$) and $d = L/k$:
(1). All parties parse $[\boldsymbol{x}]_t, [\boldsymbol{y}]_t$ as
$$[\boldsymbol{x}]_t = ([\boldsymbol{a}^{(1)}]_t, [\boldsymbol{a}^{(2)}]_t, \ldots, [\boldsymbol{a}^{(k)}]_t),$$
$$[\boldsymbol{y}]_t = ([\boldsymbol{b}^{(1)}]_t, [\boldsymbol{b}^{(2)}]_t, \ldots, [\boldsymbol{b}^{(k)}]_t)$$
where $\{\boldsymbol{a}^{(i)}, \boldsymbol{b}^{(i)}\}_{i \in [k]}$ are the vectors of dimension d.
(2). For $i \in [k-1]$, all parties invoke Π_{ExMult} on $\{[\boldsymbol{a}^{(i)}]_t, [\boldsymbol{b}^{(i)}]_t\}_{i \in [k-1]}$ to compute $[c^{(i)}]_t$ where $c^{(i)} = \boldsymbol{a}^{(i)} \odot \boldsymbol{b}^{(i)}$. Then set $[c^{(k)}]_t = [z]_t - \sum_{i=1}^{k-1}[c^{(i)}]_t$.
(3). All parties invoke Π_{ExCompr} on $\{[\boldsymbol{a}^{(i)}]_t, [\boldsymbol{b}^{(i)}]_t, [c^{(i)}]_t\}_{i=1}^k$ to get $\{[\boldsymbol{a}]_t, [\boldsymbol{b}]_t, [c]_t\}$. Then they set $\{[\boldsymbol{x}]_t, [\boldsymbol{y}]_t, [z]_t\} = \{[\boldsymbol{a}]_t, [\boldsymbol{b}]_t, [c]_t\}$.
4: **Step 4: Randomization**
Upon getting $\{[\boldsymbol{x}]_t, [\boldsymbol{y}]_t, [z]_t\}$ from Step 2, all parties do the following things.
(1). Let $L < k$ be the dimension of $\{[\boldsymbol{x}]_t, [\boldsymbol{y}]_t\}$, all parties parse $[\boldsymbol{x}]_t, [\boldsymbol{y}]_t$ as
$$[\boldsymbol{x}]_t = ([a_1]_t, [a_2]_t, \ldots, [a_L]_t)$$
$$[\boldsymbol{y}]_t = ([b_1]_t, [b_2]_t, \ldots, [b_L]_t)$$
(2). All parties execute $\mathcal{F}_{\mathsf{RandSh\text{-}Ab}}$ to prepare two random degree-t sharings $[a_0]_t, [b_0]_t$. For $i \in [0, L-1]$, all parties invoke Π_{ExMult} on $\{[a_i]_t, [b_i]_t\}_{i \in [0, L-1]}$ to compute $[c_i]_t$ where $c_i = a_i \cdot b_i$. Then set $[c^{(L)}]_t = [z]_t - \sum_{i=1}^{L-1}[c^{(i)}]_t$.

(3). All parties invoke Π_{ExCompr} on $\{[a_i]_t, [b_i]_t, [c_i]_t\}_{i=0}^{L}$ to get result $[a]_t, [b]_t, [c]_t$.
(4). For each party who has computed his shares of $[a]_t, [b]_t, [c]_t$, he sends them to all parties. Otherwise, if a party takes $\perp$ as input sharings, he sends $\perp$ to all parties.
- If a party first receive $\perp$, he sets his output decision as `Fail`.
- Otherwise, if a party first receive $2t+1$ shares, he checks whether these $2t+1$ shares of $[a]_t, [b]_t, [c]_t$ lie on a degree-t polynomial and $c = a \cdot b$. If true, he sets his output decision as `Success`; otherwise, he sets his output decision as `Fail`.

(5). All parties invoke an instance of $\mathcal{F}_{\mathsf{ba}}$ with their output decision as input. If they terminate with `Success`, they output it. Otherwise, they output `Fail`.

LEMMA E.1. *Protocol $\Pi_{\mathsf{MulTupleVrfy}}$ securely computes $\mathcal{F}_{\mathsf{MulTupleVrfy}}$ in the $\{\mathcal{F}_{\mathsf{Coin\text{-}Ab}}, \mathcal{F}_{\mathsf{RandSh\text{-}Ab}}, \mathcal{F}_{\mathsf{RandShZero\text{-}Ab}}\}$-hybrid model against a fully malicious adversary $\mathcal{A}$ who corrupts at most $t < n/3$ parties.*

We prove Lemma E.1 and analyze the costs in the following.

E.1 Cost Analysis

Communication Complexity. We compute the cost for preparing the random challenges by $\mathcal{F}_{\mathsf{Coin\text{-}Ab}}$ in the end. In step 1, all parties need to prepare $O(N')$ random degree-t and $2t$ sharings, which costs $O(n \log N + n^3)$ field elements. In step 2, all parties only perform local computation. In step 3, all parties invoke $\Pi_{\mathsf{ExMult}}, \Pi_{\mathsf{ExCompr}}$ in each iteration, and there are $\log N$ interactions in total, which costs $O(n^2 \log N)$ field elements in total. In step 4, similar to step 3, but there is only one iteration, which costs $O(n^2)$ field elements in total.

For $\mathcal{F}_{\mathsf{Coin\text{-}Ab}}$, all parties need to generate $O(\log N)$ random challenges in total during the whole process, which costs $O(n^2 \log N)$ field elements. The $\mathcal{F}_{\mathsf{ba}}$ can be realized with $O(n^3)$ field elements. Therefore, the total communication complexity is $O(n^2 \log N + n^3)$ field elements.

Round Complexity. The expected round complexity of all building blocks $\mathcal{F}_{\mathsf{Coin\text{-}Ab}}, \mathcal{F}_{\mathsf{ba}}, \mathcal{F}_{\mathsf{RandSh\text{-}Ab}}, \mathcal{F}_{\mathsf{RandShZero\text{-}Ab}}$ is constant. Step 3 requires $\log N$ interactions, and the round complexity of each iteration is constant. Therefore, the whole round complexity is $O(\log N)$.

E.2 Security Analysis

We prove lemma E.1 and start with constructing the ideal adversary $\mathcal{S}$ as follows.

Simulator $\mathcal{S}_{\mathsf{ExMult}}$

$\mathcal{S}$ takes corrupted parties' shares of $\{[\boldsymbol{a}^{(\ell)}]_t, [\boldsymbol{b}^{(\ell)}]_t, [r]_t^{(\ell)}\}_{\ell=1}^{N}$ and $\{[o]_t^{(\ell)}\}_{\ell=1}^{N/2}$ as inputs. For each $[o]_t^{(\ell)}$, $\mathcal{S}$ takes an additive error $\boldsymbol{e}_\ell$ as input.

1: $\mathcal{S}$ computes corrupted parties' shares of $([\tilde{o}_1]_{2t}, \ldots, [\tilde{o}_n]_{2t})$. For $\ell \in [t]$, $\mathcal{S}$ also computes the corresponding additive errors $(\tilde{\boldsymbol{e}}_1, \ldots, \tilde{\boldsymbol{e}}_n)$ as follows:
$$(\tilde{\boldsymbol{e}}_1, \ldots, \tilde{\boldsymbol{e}}_n) = \boldsymbol{M} \cdot (\boldsymbol{e}_1, \ldots, \boldsymbol{e}_t).$$
2: $\mathcal{S}$ first computes corrupted parties' shares of $\{[z_k]_{2t}\}_{k=1}^{2t+1}$ and $\{[f(\alpha_i)]_{2t}\}_{i=1}^{n}$, then he randomly samples the whole sharing $[\tilde{f}(\alpha_i)]_{2t}$ for each $i \in [n]$ based on corrupted parties' share of $[f(\alpha_i)]_{2t} + [\tilde{o}_i]_{2t}$. Denote the jth value of $[\tilde{f}(\alpha_i)]_{2t}, \tilde{\boldsymbol{e}}_i$ as $\tilde{f}_i^{(j)}, \tilde{e}_i^{(j)}$, when an honest party P_j needs to distribute his share of $[\tilde{f}(\alpha_i)]_{2t}$ to P_i, $\mathcal{S}$ sends $\tilde{f}_i^{(j)} + \tilde{e}_i^{(j)}$ to P_i on behalf of honest party P_j.
3: $\mathcal{S}$ honestly follows the protocol to execute each honest party P_i and learns his reconstruction output. If the output is $\perp$, $\mathcal{S}$ sends $Z^{(i)} = \perp$ to all parties. Otherwise, $\mathcal{S}$ computes the hash of the reconstruction outputs as $Z^{(i)}$ and sends it to all parties.
4: For an honest party P_i, if his output is `Fail`, $\mathcal{S}$ records it. If P_i receives the same $Z^{(j)}$ from $2t+1$ distinct P_j, $\mathcal{S}$ records the underlying reconstruction output $\{\tilde{z}_k\}_{k=1}^{2t+1}$. Recall that $\mathcal{S}$ has samples $\tilde{f}(\alpha_i)$ for all $i \in [n]$ in step (2), then he can compute the real $\{z_k\}_{k=1}^{2t+1}$. Finally $\mathcal{S}$ computes and records the differences $\tilde{z}_k - z_k$ for each $k \in [2t+1]$. If at least one of them is not 0, $\mathcal{S}$ sets a flag as `Error`.
5: $\mathcal{S}$ computes corrupted parties' shares of $[c_k]_t$. Then he terminates with the additive errors and shares of corrupted parties.

Simulator $\mathcal{S}_{\mathsf{ExCompress}}$

$\mathcal{S}$ takes corrupted parties' shares of $\{[\boldsymbol{x}^{(i)}]_t, [\boldsymbol{y}^{(i)}]_t, [z]_t^{(i)}\}_{i=1}^{N}$ and additive errors $\{e^{(i)}\}_{i=1}^{N}$ such that each $e^{(i)} = z^{(i)} - \boldsymbol{x}^{(i)} \odot \boldsymbol{y}^{(i)}$ as inputs.

1: For $i \in [N+1, 2N-1]$, $\mathcal{S}$ follows the protocol to compute corrupted parties' shares of $[\boldsymbol{f}(\alpha_i)]_t, [\boldsymbol{g}(\alpha_i)]_t$.
2: $\mathcal{S}$ invokes $\mathcal{S}_{\mathsf{ExMult}}$ and learns corrupted parties' shares of $[z^{(i)}]_t$ and an additive error $e^{(i)} = z^{(i)} - \boldsymbol{f}(\alpha_i) \odot \boldsymbol{g}(\alpha_i)$ for $i \in [N+1, 2N-1]$.
3: $\mathcal{S}$ sets a degree-$2(N-1)$ polynomial $e(\cdot)$ such that $d(\alpha_i) = e^{(i)}$ for $i \in [1, 2N-1]$.
4: $\mathcal{S}$ honestly emulates $\mathcal{F}_{\mathsf{Coin\text{-}Ab}}$ and learns a random element r. For each honest party, if $r \in [\alpha_1, \ldots, \alpha_N]$ or his output of $\mathcal{F}_{\mathsf{Coin\text{-}Ab}}$ is `Fail`, $\mathcal{S}$ records it. Then $\mathcal{S}$ terminates with corrupted parties' shares and an additive error $e(r)$.

Simulator $\mathcal{S}$

Let N denote the number of multiplication tuples to be verified.

1: $\mathcal{S}$ invokes $\mathcal{F}_{\mathsf{MulTupleVrfy}}$ and learns corrupted parties' shares of $\{[a^{(\ell)}]_t, [b^{(\ell)}]_t, [c^{(\ell)}]_t\}_{\ell=0}^{N}$ and additive errors $\{d^{(\ell)}\}_{\ell \in [N]}$.
2: In Step 1, $\mathcal{S}$ honestly simulates $\mathcal{F}_{\mathsf{RandSh\text{-}Ab}}$ and $\mathcal{F}_{\mathsf{RandShZero\text{-}Ab}}$:
- During the simulation of $\mathcal{F}_{\mathsf{RandSh\text{-}Ab}}$, $\mathcal{S}$ receives shares of corrupted parties and learns which honest party's output is `Fail` or `abort`.
- During the simulation of $\mathcal{F}_{\mathsf{RandShZero\text{-}Ab}}$, $\mathcal{S}$ receives shares of corrupted parties, an additive error for each sharing, and learns which honest party's output is `Fail`.

3: In Step 2, $\mathcal{S}$ honestly emulates $\mathcal{F}_{\mathsf{Coin\text{-}Ab}}$ and learns random value r, then it computes corrupted parties' shares of $[f]_t, [g]_t$ and $[h]_t$. For each honest party whose output of $\mathcal{F}_{\mathsf{Coin\text{-}Ab}}$ is `Fail`, $\mathcal{S}$ records it. Then, in the following simulation of this honest party, $\mathcal{S}$ will replace his message with the default message $\perp$.
4: In Step 3, $\mathcal{S}$ does the following. Let e be an additive error such that $e = z - \boldsymbol{x} \odot \boldsymbol{y}$, which will be initialized to $e = \sum_{\ell=1}^{N} r^{\ell-1} \cdot d^{(\ell)}$.
(1). For $i \in [k-1]$, $\mathcal{S}$ computes corrupted parties' shares of $[\boldsymbol{a}^{(i)}]_t, [\boldsymbol{b}^{(i)}]_t$. Then $\mathcal{S}$ invokes $\mathcal{S}_{\mathsf{ExMult}}$ and learns corrupted parties' shares of $[c^{(i)}]$ and the additive error $e^{(i)} = c^{(i)} - \boldsymbol{a}^{(i)} \odot \boldsymbol{b}^{(i)}$.

(2). $\mathcal{S}$ follows the protocol to compute corrupted parties' shares of $[c^{(k)}]_t$ and the additive error $e^{(k)} = e - \sum_{i=1}^{k-1} e^{(i)}$. Then he invokes $\mathcal{S}_{\mathsf{ExCompress}}$ and learns corrupted parties' shares of $\{[\boldsymbol{a}]_t, [\boldsymbol{b}]_t, [c]_t\}$ and an additive error $e' = c - \boldsymbol{a} \odot \boldsymbol{b}_t$.

(3). $\mathcal{S}$ updates $e \leftarrow e'$ and moves to the next rounds.

5: In Step 4, $\mathcal{S}$ does the same thing as above to do simulation and learns corrupted parties' shares of $\{[a]_t, [b]_t, [c]_t\}$ and an additive error $e = c - a \cdot b$ in the end. Then $\mathcal{S}$ first randomly samples values as a, b, then randomly samples the whole sharing $[a]_t, [b]_t$ and $[c]_t$ based on shares of corrupted parties and secrets a, b and $a \cdot b + e$. Then $\mathcal{S}$ honestly follows the protocol to execute each honest party.

6: For each honest party P_i whose output should be $\mathtt{Fail}$, $\mathcal{S}$ sends $(\mathtt{Fail}, P_i)$ to $\mathcal{F}_{\mathsf{MulTupleVrfy}}$.

The hybrid arguments are as follows.

$\mathbf{Hyb}_0$: In this hybrid, we consider the execution in the real world.

$\mathbf{Hyb}_1$: In this hybrid, $\mathcal{S}$ computes the difference for each multiplication tuple and inner-product tuple as described above. Note that this does not change the behaviors of honest parties. Therefore, the distribution of $\mathbf{Hyb}_1$ and $\mathbf{Hyb}_0$ is identical.

$\mathbf{Hyb}_2$: In this hybrid, instead of using the real sharing $[a]_t, [b]_t, [c]_t$ in Step 4: Randomization, $\mathcal{S}$ will construct the sharings $[a]_t, [b]_t, [c]_t$ as described above. Note that in Step 4: Randomization, a, b are linear combinations of $\{a_i\}_{i=0}^{L}, \{b_i\}_{i=0}^{L}$ respectively, and the coefficients are all non-zero, where the latter follows from the property of polynomials. Also note that a_0, b_0 are randomly chosen by $\mathcal{F}_{\mathsf{RandSh\text{-}Ab}}$. Thus, a and b are uniformly random. The only difference between $\mathbf{Hyb}_2$ and $\mathbf{Hyb}_1$ is that, in $\mathbf{Hyb}_1$, a and b are masked by a_0 and b_0 which are randomly chosen by $\mathcal{F}_{\mathsf{RandSh\text{-}Ab}}$, while in $\mathbf{Hyb}_2$, a and b are randomly chosen by $\mathcal{S}$. The distributions of a, b remain unchanged. Since c is determined by a, b and the difference d', the distribution of c remains the same. Therefore, the distribution of $\mathbf{Hyb}_2$ and $\mathbf{Hyb}_1$ is identical.

$\mathbf{Hyb}_3$: In this hybrid, $\mathcal{S}$ will abort if at least one of the input multiplication tuples is incorrect. Note that in the real protocol, it is possible that while one of the input multiplication tuples is incorrect, the final multiplication tuple verified in Randomization is correct. In this case, honest parties in $\mathbf{Hyb}_2$ do not abort. According to the analysis in [GS20], the probability is negligible in the security parameter. Therefore, the distribution of $\mathbf{Hyb}_3$ is statistically close to $\mathbf{Hyb}_2$.

$\mathbf{Hyb}_4$: In this hybrid, In this hybrid, $\mathcal{S}$ simulates the behaviors of honest parties as described above. The only place honest parties need to communicate with corrupted parties is that they exchange their shares of $[a]_t, [b]_t, [c]_t$. However, the preparation of $[a]_t, [b]_t, [c]_t$ only depends on the shares and the differences of the input multiplication tuples, which can be obtained from $\mathcal{F}_{\mathsf{MulTupleVrfy}}$. Therefore, the distribution of $\mathbf{Hyb}_4$ and $\mathbf{Hyb}_3$ is identical.

Note that $\mathbf{Hyb}_4$ corresponds to the ideal world, then $\Pi_{\mathsf{MulTupleVrfy}}$ securely computes $\mathcal{F}_{\mathsf{MulTupleVrfy}}$.

F CONSTRUCTION AND ANALYSIS OF AMPC

In the following, we give the construction of our fairness AMPC protocol.

F.1 Construction

We first realize a security-with-abort MPC protocol by the following four phases. Roughly speaking, all parties will generate *double sharing* in the offline phase, which will be used in Π_{Eval} to evaluate the circuit in the online phase. After the verification phase, if all parties output $\mathtt{Fail}$, they can abort the protocol. Otherwise, the computing result is correct with overwhelming probability, and all parties will proceed. Note that if all parties proceed, some honest parties may do not have their shares of the output wires.

Protocol $\Pi_{\mathsf{Main\text{-}Fair}}$

Offline Phase

1: **Initialization of Private Channel.** For each pair of parties (P_i, P_j), all parties initialize an instance of $\mathcal{F}_{\mathsf{PrivSend\text{-}Ab}}$. For each party P_i, when all parties receives $(\mathtt{Delivered}, \mathtt{Init})$ from all instances of $\mathcal{F}_{\mathsf{PrivSend\text{-}Ab}}$ between P_i and P_j, they continue to allow P_i to participate in the instances of $\mathcal{F}_{\mathsf{RandSh\text{-}Ab}}$ and $\mathcal{F}_{\mathsf{RandShZero\text{-}Ab}}$.

2: **Preparation of Random Sharings.** Let C denote the circuit to be computed. All parties invoke $\mathcal{F}_{\mathsf{RandSh\text{-}Ab}}$ to prepare $|C|$ random degree-t Shamir sharings and $\mathcal{F}_{\mathsf{RandShZero\text{-}Ab}}$ to prepare $|C|/2$ random degree-$2t$ Shamir sharings of zero. If a party receives $\mathtt{Fail}$ from $\mathcal{F}_{\mathsf{RandSh\text{-}Ab}}$ or $\mathcal{F}_{\mathsf{RandShZero\text{-}Ab}}$, he sets his output as $\perp$ and proceeds.

Input Phase

1: Each party P_i acts as a dealer invokes $\mathcal{F}_{\mathsf{ACSS\text{-}Ab}}$ to distribute his input x_i.

2: Each party P_i sets the property Q as P_i terminating $\mathcal{F}_{\mathsf{ACSS\text{-}Ab}}$ where the dealer is P_j. Then all parties invoke $\mathcal{F}_{\mathsf{acs}}$ with property Q to agree on a set $\mathcal{D}$ of size $2t + 1$ and each party in this set has successfully shared their inputs. For every $P_i \notin \mathcal{D}$, all parties set their shares of P_i's input as 0.

3: For each dealer in $\mathcal{D}$, if all parties terminate $\mathcal{F}_{\mathsf{ACSS\text{-}Ab}}$ led by this dealer with $\mathtt{Fail}$, they terminate with $\mathtt{Fail}$. Otherwise, they proceed.

Computation Phase

1: The circuit is partitioned into a sequence of two-layer sub-circuits. All sub-circuits are evaluated in a predetermined topological order. For each sub-circuit with all the input sharings prepared, all parties invoke Π_{Eval} to compute the output sharings.

Verification Phase

1: All parties invoke $\mathcal{F}_{\mathsf{MulTupleVrfy}}$ to check the correctness of the multiplications. If a party receives $\mathtt{Fail}$ from $\mathcal{F}_{\mathsf{MulTupleVrfy}}$, he sets his output as $\perp$ and proceeds. Otherwise, he sets his output as all his shares of output sharing and proceeds.

To achieve fairness AMPC, all parties continue to execute the following two phases. In the output phase, they first prepare a random mask $[r]_t$, which will be added to the output wire $[y]_t$, then exchange their shares of $[y + r]_t$ and use online error correction (OEC) to reconstruct $y + r$. In the termination phase, all parties will execute a non-intrusion BA protocol to agree on the reconstruction result. Here, the non-intrusion BA is realized by two instances of $\mathcal{F}_{\mathrm{ba}}$, which can also be replaced with other constructions, e.g., [MR15]. If they succeed in reconstructing $y + r$, they will further reconstruct the mask r and then recover the secret y.

Protocol $\Pi_{\texttt{Main-Fair}}$

Output Phase

1: Let C_O be the output size of C, all parties invoke $\mathcal{F}_{\text{RandSh-Ab}}$ to prepare C_O random degree-t Shamir secret sharing. If all parties terminate $\mathcal{F}_{\text{RandSh-Ab}}$ with `Fail`, they terminate with output `Fail`. Otherwise, they proceed.
2: All parties assign an unused random degree-t sharing $[r]_t$ for each output wire $[y]_t$. For each party that can compute his share of $[y+r]_t = [y]_t + [r]_t$, he will send it to all parties. Otherwise, he sets his output as $\perp$, sends $\perp$ to all parties and proceeds.
3: For each party, if he first receives $2t+1$ shares of $[y+r]_t$ and these shares lie on a degree-t polynomial, he reconstructs and outputs the secret $y+r$. Otherwise, if he first receives $\perp$ or fails to do reconstruction, he sets his output as $\perp$ and proceeds.

Termination Phase.

1: All parties invoke $\mathcal{F}_{\text{ba}}$ to agree on the output. For each party P_i, if his output is $y' = y + r$, he sets $y_i = 1|y'$. Otherwise, if his output has been set to $\perp$, he sets y_i as a zero string of size $|y'|+1$. Then he sends y_i to $\mathcal{F}_{\text{ba}}$.
2: Upon receiving $\bar{y}$ from $\mathcal{F}_{\text{ba}}$, if the first bit of $\bar{y}$ is 1, all parties proceed. Otherwise, all parties terminate with output `Fail`.
3: Each party P_i checks whether $\bar{y} = y_i$. If true, P_i sets $b_i = 1$; otherwise, he sets $b_i = 0$. Then all parties invoke another $\mathcal{F}_{\text{ba}}$ to agree on whether the output is valid (meaning that the output comes from an honest party). Each party P_i sends b_i to $\mathcal{F}_{\text{ba}}$.
4: Upon receiving b from $\mathcal{F}_{\text{ba}}$, if $b = 0$, all parties terminate with output `Fail`. Otherwise, for each random degree-t sharing $[r]_t$ used to mask the output wire $[y]_t$, all parties send request `Public-Recon` to $\mathcal{F}_{\text{RandSh-Ab}}$ (the one in the step 1 of output phase) and receive secret r. Then all parties terminate with output $y = \bar{y} - r$.

LEMMA F.1. *Protocol $\Pi_{\texttt{Main-Fair}}$ securely computes $\mathcal{F}_{\text{AMPC-Fair}}$ in the $\{\mathcal{F}_{\text{ACSS-Ab}}, \mathcal{F}_{\text{acs}}, \mathcal{F}_{\text{RandSh-Ab}}, \mathcal{F}_{\text{RandShZero-Ab}}, \mathcal{F}_{\text{MulTupleVrfy}}\}$-hybrid model against a fully malicious adversary $\mathcal{A}$ who corrupts at most $t < n/3$ parties.*

We prove Lemma F.1 and analyze the costs in the following.

F.2 Cost Analysis

We analyze the communication and round complexity as follows.

Communication Complexity. We analyze the communication cost of each phase as follows.

- In the offline phase, all parties need to initialize all $\mathcal{F}_{\text{PrivSend-Ab}}$, which requires $O(n^3)$ field elements. For the preparation of $|C|$ degree-t random sharing and $|C|/2$ degree-$2t$ random sharing of 0, it costs $O(|C| \cdot n + n^3)$ field elements.
- In the input phase, all parties first invoke $\mathcal{F}_{\text{ACSS-Ab}}$ to distribute their inputs, which requires $O(n^3)$ field elements. Then they invoke $\mathcal{F}_{\text{acs}}$ to agree on a set of successful dealers, which requires $O(n^3)$ field elements.
- In the computation phase, all parties evaluate every two-layer sub-circuit one by one. It costs $O(|C| \cdot n + D \cdot n^2)$ field elements in total.
- In the verification phase, all parties execute $\mathcal{F}_{\text{MulTupleVrfy}}$, which costs $O(n^2 \cdot \log|C| + n^3)$ field elements in total.
- In the output and termination phase, it costs $O(C_O \cdot n^2 + n^3)$ field elements in total.

Therefore, the total communication complexity is $O(|C| \cdot n + D \cdot n^2 + n^3)$ field elements.

Round Complexity. Since the expected round complexity of building blocks $\mathcal{F}_{\text{PrivSend-Ab}}, \mathcal{F}_{\text{RandSh-Ab}}, \mathcal{F}_{\text{RandShZero-Ab}}, \mathcal{F}_{\text{ACSS-Ab}}, \mathcal{F}_{\text{acs}},, \mathcal{F}_{\text{ba}}$ are all constant, then the round complexity of the offline phase, input phase, output phase, and termination phase is constant. In the computation phase, since the circuit depth is D, and all parties require 3 rounds to evaluate every two-layer sub-circuit, the round complexity will be $\lceil \frac{D}{2} \rceil \cdot 3 = O(D)$. In the verification phase, it costs $\log|C|$ depth for verifying $|C|$ multiplication gates. Therefore, the total expected round complexity is $O(D + \log|C|)$.

F.3 Security Analysis

We prove lemma F.1 and start with constructing the ideal adversary $\mathcal{S}$ as follows.

Simulator $\mathcal{S}$

Denote $|C|$ as the number of Beaver triples. Let $Corr$ denote the set of corrupted parties, then $|Corr| = t' \le t$. Let $Corr'$ be the set of all corrupted parties together with the first $t - t'$ honest parties, then $|Corr'| = t$

1: In the offline phase, $\mathcal{S}$ honestly simulates $\mathcal{F}_{\text{RandSh-Ab}}$ and $\mathcal{F}_{\text{RandShZero-Ab}}$:
   - During the simulation of $\mathcal{F}_{\text{RandSh-Ab}}$, $\mathcal{S}$ receives shares of corrupted parties and learns which honest party's output is `Fail` or `abort`.
   - During the simulation of $\mathcal{F}_{\text{RandShZero-Ab}}$, $\mathcal{S}$ receives shares of corrupted parties, an additive error for each sharing, and learns which honest party's output is `Fail`.

   For each honest party in $Corr' \setminus Corr$, $\mathcal{S}$ samples random values as his shares.
2: In the input phase, $\mathcal{S}$ simulates each $\mathcal{F}_{\text{ACSS-Ab}}$ as follows:
   - For each honest party, for all parties in $Corr'$, $\mathcal{S}$ randomly samples values as their shares. Then $\mathcal{S}$ sends corrupted parties' shares to parties in $Corr$ on behalf of $\mathcal{F}_{\text{ACSS-Ab}}$.
   - For each corrupted party, $\mathcal{S}$ waits to receive degree-t Shamir secret sharings from him and honestly follows $\mathcal{F}_{\text{ACSS-Ab}}$. For each honest party in $Corr' \setminus Corr$, $\mathcal{S}$ computes his input shares.

   Then $\mathcal{S}$ simulates $\mathcal{F}_{\text{acs}}$ and learn a set $\mathcal{D}$ of size $2t+1$. For each honest party, if his output of $\mathcal{F}_{\text{ACSS-Ab}}$ for any dealer in $\mathcal{D}$ is `abort`, $\mathcal{S}$ sets this honest party's output as $\perp$. If his output is `Fail`, $\mathcal{S}$ sends `Fail` to $\mathcal{F}_{\text{AMPC-Fair}}$.
3: In the computation phase, during the simulation of Π_{Eval}, $\mathcal{S}$ first computes and transforms shares of parties in $Corr'$ to Beaver-triple friendly form, then simulates $\Pi_{\texttt{Mult}}$ as follows. For every group of $2t+1$ multiplication gates:

(1). For each party in $Corr'$, $\mathcal{S}$ computes his shares of $([\tilde{o}_1]_{2t}, \ldots, [\tilde{o}_n]_{2t})$. For $\ell \in [t]$, denote the additive error for each $[o_\ell]_{2t}$ is $\boldsymbol{e}_\ell$, $\mathcal{S}$ also computes the corresponding additive errors $(\tilde{\boldsymbol{e}}_1, \ldots, \tilde{\boldsymbol{e}}_n)$ as follows:

$$(\tilde{\boldsymbol{e}}_1, \ldots, \tilde{\boldsymbol{e}}_n) = \boldsymbol{M} \cdot (\boldsymbol{e}_1, \ldots, \boldsymbol{e}_t).$$

(2). $\mathcal{S}$ first computes shares of $\{[z_k]_{2t}\}_{k=1}^{2t+1}$ and $\{[f(\alpha_i)]_{2t}\}_{i=1}^{n}$ for each party in $Corr'$, then he randomly samples the whole sharing $[\tilde{f}(\alpha_i)]_{2t}$ for each $i \in [n]$ based on P_j's share of $[f(\alpha_i)]_{2t} + [\tilde{o}_i]_{2t}$, where $j \in Corr'$. Denote the jth value of

$[\tilde{f}(\alpha_i)]_{2t}, \tilde{\boldsymbol{e}}_i$ as $\tilde{f}_i^{(j)}, \tilde{e}_i^{(j)}$, when an honest party P_j needs to distribute his share of $[\tilde{f}(\alpha_i)]_{2t}$ to P_i, $\mathcal{S}$ sends $\tilde{f}_i^{(j)} + \tilde{e}_i^{(j)}$ to P_i on behalf of honest party P_j.

(3). $\mathcal{S}$ honestly follows the protocol to execute each honest party P_i and learns his reconstruction output. If the output is $\perp$, $\mathcal{S}$ sends $Z^{(i)} = \perp$ to all parties. Otherwise, $\mathcal{S}$ computes the hash of the reconstruction outputs as $Z^{(i)}$ and sends it to all parties.

(4). For an honest party P_i, if his output is `Fail`, $\mathcal{S}$ records it. If P_i receives the same $Z^{(j)}$ from $2t+1$ distinct P_j, $\mathcal{S}$ records the underlying reconstruction output $\{\tilde{z}_k\}_{k=1}^{2t+1}$. Recall that $\mathcal{S}$ has samples $\tilde{f}(\alpha_i)$ for all $i \in [n]$ in step (2), then he can compute the real $\{z_k\}_{k=1}^{2t+1}$. Finally $\mathcal{S}$ computes and records the differences $\tilde{z}_k - z_k$ for each $k \in [2t+1]$. If at least one of them is not 0, $\mathcal{S}$ sets a flag as `Error`.

(5). $\mathcal{S}$ computes shares of $[c_k]_t$ for all parties in $Corr'$.

When $\mathcal{S}$ terminates the simulation of $\Pi_{\texttt{Mult}}$:

- If $\mathcal{S}$ learns the public reconstruction output, $\mathcal{S}$ follows the protocol to compute shares of $[z_t]$ for parties in $Corr'$. Recall that for the public reconstruction output, $\mathcal{S}$ has computed the additive errors during the simulation of $\Pi_{\texttt{Mult}}$:
  - For multiplication gates in the first layer, $\mathcal{S}$ can compute the final additive errors for these multiplication tuples based on the errors in the public reconstruction output.
  - For multiplication gates in the second layer, since there exist terms $u \cdot [b]_t$ and $v \cdot [a]_t$, and the secrets b, a are not known to $\mathcal{S}$. He can not extract the final additive errors for these multiplication tuples (causes issues in the simulation of $\mathcal{F}_{\mathsf{MulTupleVrfy}}$ later). To deal with it, recall that if $\mathcal{S}$ has recorded a flag `Error`, then $\mathcal{S}$ will send `Fail` to $\mathcal{F}_{\mathsf{AMPC\text{-}Fair}}$ and the adversary will learn nothing about the output. So $\mathcal{S}$ can randomly sample the whole sharing $[a]_t, [b_t]$ based on shares of parties in $Corr'$ and then extracts the additive errors.
- Otherwise, it implies that all honest parties' outputs are `Fail`, then in the following simulation, $\mathcal{S}$ can directly follow the protocol to execute each honest party.

4: In the verification phase, $\mathcal{S}$ honestly follows the protocol. If all parties receive `Stop` from $t+1$ distinct parties' broadcast, $\mathcal{S}$ sends `Fail` to $\mathcal{F}_{\mathsf{AMPC\text{-}Fair}}$. Otherwise, $\mathcal{S}$ honestly simulates $\mathcal{F}_{\mathsf{MulTupleVrfy}}$ as follows:

- Recall that in the simulation of the computation phase, $\mathcal{S}$ has computed the shares of each multiplication tuple held by parties in $Corr'$ and the difference. Then it sends these shares and differences to the adversary as in $\mathcal{F}_{\mathsf{MulTupleVrfy}}$.
- If $\mathcal{S}$ has recorded a flag `Error`, $\mathcal{S}$ sets b = `Fail` and `Success` otherwise. Then $\mathcal{S}$ sends b to the adversary.
  - If b = `Success` and the adversary replies nothing, $\mathcal{S}$ continues to the next phase.
  - If b = `Fail` or the adversary replies `Fail`, $\mathcal{S}$ sends `Fail` to $\mathcal{F}_{\mathsf{AMPC\text{-}Fair}}$.

5: In the output phase, $\mathcal{S}$ honestly simulates $\mathcal{F}_{\mathsf{RandSh\text{-}Ab}}$ and learns shares of corrupted parties and which honest party's output is `Fail` or `abort`. For each honest party in $Corr'$, $\mathcal{S}$ randomly samples a value as his shares. For each party in $Corr'$, $\mathcal{S}$ locally computes his share of $[y+r]_t$. Then $\mathcal{S}$ randomly samples the whole sharing based on the shares of parties in $Corr'$. With the whole sharing $[y+r]_t$, $\mathcal{S}$ honestly follows the rest of the steps.

Termination Phase

6: In the termination phase, $\mathcal{S}$ honestly simulates these two $\mathcal{F}_{\mathsf{ba}}$ and follows the protocol to determine all honest parties' output is $\bar{y}$ or `Fail`. If the output is `Fail`, $\mathcal{S}$ sends `Fail` to $\mathcal{F}_{\mathsf{AMPC\text{-}Fair}}$. Otherwise, if the output is $\bar{y}$, $\mathcal{S}$ sends the inputs of corrupted parties and the set $\mathcal{D}$ to $\mathcal{F}_{\mathsf{AMPC\text{-}Fair}}$ and receives the output y. Then $\mathcal{S}$ computes the whole sharing $[y]_t$ based on shares of parties in $Corr'$ and then computes $[r]_t = [y+r]_t - [y]_t$. With the whole sharing $[r]_t$, $\mathcal{S}$ gets the secrets r and sends them to corrupted parties on behalf of $\mathcal{F}_{\mathsf{RandSh\text{-}Ab}}$.

7: $\mathcal{S}$ outputs the views of $\mathcal{A}$.

The hybrid arguments are as follows.

$\mathbf{Hyb}_0$: In this hybrid, we consider the execution in the real world.

$\mathbf{Hyb}_1$: In this hybrid, when $\mathcal{S}$ receives degree-t Shamir sharings from a corrupted dealer, $\mathcal{S}$ records the first $t - t'$ honest parties' shares. $\mathbf{Hyb}_1$ and $\mathbf{Hyb}_0$ have the same distribution.

$\mathbf{Hyb}_2$: In this hybrid, in the input phase, for each honest dealer, after randomly sampling shares of parties in $Corr'$, $\mathcal{S}$ delays the generation of shares of the rest of the honest parties until the set $\mathcal{D}$ is determined. Since these honest parties' shares are not used in the input phase, $\mathbf{Hyb}_2$ and $\mathbf{Hyb}_1$ have the same distribution.

$\mathbf{Hyb}_3$: In this hybrid, in the input phase, for each honest dealer not in $\mathcal{D}$, $\mathcal{S}$ does not generate shares of honest parties. Since these honest parties' sharings are never used, $\mathbf{Hyb}_3$ and $\mathbf{Hyb}_2$ have the same distribution.

$\mathbf{Hyb}_4$: In this hybrid, in the computation phase, during the simulation of $\Pi_{\texttt{Mult}}$, we change the generation of $[o_1]_{2t}, \ldots, [o_t]_{2t}$. For each $[\tilde{o}_i]_{2t}$ which will be used for parties in $Corr'$, $\mathcal{S}$ randomly samples the whole sharing based on shares of corrupted parties. Then $\mathcal{S}$ recomputes $\{[o_\ell]_{2t}\}_{\ell=1}^{t}$ from $\{[\tilde{o}_i]_{2t}\}_{i\in Corr'}$, then computes $\{[\tilde{o}_i]_{2t}\}_{i\notin Corr'}$. Since there is a one-to-one map between $\{[o_\ell]_{2t}\}_{\ell=1}^{t}$ and $\{[\tilde{o}_i]_{2t}\}_{i\in Corr'}$, this adjustment makes no difference. Therefore, $\mathbf{Hyb}_4$ and $\mathbf{Hyb}_3$ have the same distribution.

$\mathbf{Hyb}_5$: In this hybrid, in the computation phase, during the simulation of $\Pi_{\texttt{Mult}}$, we further change the generation of $\{[\tilde{o}_i]_{2t}\}_{i\in Corr'}$ by letting the $\mathcal{S}$ first samples the whole sharing $[\tilde{f}(\alpha_i)]_{2t}$ which will be reconstructed to corrupted parties, then recompute each $[\tilde{o}_i]_{2t} = [\tilde{f}(\alpha_i)]_{2t} - [f(\alpha_i)]_{2t}$. Since both of them are randomly sampled, $\mathbf{Hyb}_5$ and $\mathbf{Hyb}_4$ have the same distribution.

$\mathbf{Hyb}_6$: In this hybrid, in the computation phase, during the simulation of $\Pi_{\texttt{Mult}}$, $\mathcal{S}$ changes the generation of $[r_k]_t$ by first sampling a random value as z_k, then computing $r_k = z_k - a_k \cdot b_k$. Then $\mathcal{S}$ samples the whole sharing $[r_k]_t$ based on the secrets and shares of parties in $Corr'$. Since both r_k and z_k are randomly sampled, $\mathbf{Hyb}_6$ and $\mathbf{Hyb}_5$ have the same distribution.

$\mathbf{Hyb}_7$: In this hybrid, in the computation phase, during the simulation of $\Pi_{\texttt{Mult}}$, $\mathcal{S}$ no longer use honest parties' shares to compute their shares of $[\tilde{f}(\alpha_i)]_{2t}$ used for corrupted party P_i. $\mathcal{S}$ extracts $\tilde{\boldsymbol{e}}_i$ as above, then $\mathcal{S}$ sends $\tilde{f}_i^{(j)} + \tilde{e}_i^{(j)}$ to P_i on behalf of honest party P_j. $\mathbf{Hyb}_7$ and $\mathbf{Hyb}_6$ have the same distribution.

$\mathbf{Hyb}_8$: In this hybrid, in the computation phase, during the simulation of $\Pi_{\texttt{Mult}}$, $\mathcal{S}$ changes the computation of $\tilde{f}(\alpha_i)$ for honest P_i. Instead, $\mathcal{S}$ has sampled $z_1, \ldots, z_{2t+1}$ and already know $f(X)$, he can first computes $f(\alpha_i)$. Based on the shares of $[\tilde{f}(\alpha_i)]_{2t}$ received from corrupted parties, $\mathcal{S}$ extracts the error e_i, then computes honest P_i's $\tilde{f}(\alpha_i) = f(\alpha_i) + e_i$. $\mathbf{Hyb}_8$ and $\mathbf{Hyb}_7$ have the same distribution.

$\mathbf{Hyb}_9$: In this hybrid, in the computation phase, during the simulation of $\Pi_{\texttt{Mult}}$, if an honest party accepts his reconstruction result, but

not the same as other honest parties' reconstruction results, $\mathcal{S}$ aborts the simulation. This will happen if the adversary finds a collision, which is negligible in the security parameter. The distribution of $\mathbf{Hyb}_9$ and $\mathbf{Hyb}_8$ is computationally indistinguishable.

$\mathbf{Hyb}_{10}$: In this hybrid, in the output phase, $\mathcal{S}$ delays the computation of honest parties' output wire $[y]_t$. He first samples values as $y + r$, then samples the whole sharing $[y + r]_t$ based on shares of parties in $Corr'$. When an honest party needs to send his shares of $[y + r]_t$ to corrupted parties, $\mathcal{S}$ replaces it with the shares sampled by himself. $\mathbf{Hyb}_{10}$ and $\mathbf{Hyb}_9$ have the same distribution.

$\mathbf{Hyb}_{11}$: In this hybrid, in the termination phase, when all parties agree on $y + r$ after two instances of $\mathcal{F}_{\mathsf{ba}}$, $\mathcal{S}$ invokes $\mathcal{F}_{\mathsf{AMPC\text{-}Fair}}$ and learns output y. Then $\mathcal{S}$ compute $r = y + r - y$, and samples the whole sharing $[r]_t$ based on secret r and shares of parties in $Corr'$. $\mathbf{Hyb}_{11}$ and $\mathbf{Hyb}_{10}$ have the same distribution.

$\mathbf{Hyb}_{12}$: In this hybrid, $\mathcal{S}$ no longer requires honest parties' inputs during the simulation. He also let $\mathcal{F}_{\mathsf{AMPC\text{-}Fair}}$ deliver the output to all parties. $\mathbf{Hyb}_{12}$ and $\mathbf{Hyb}_{11}$ have the same distribution.

Note that $\mathbf{Hyb}_{12}$ corresponds to the ideal world, then $\Pi_{\mathtt{Main\text{-}Fair}}$ securely computes $\mathcal{F}_{\mathsf{AMPC\text{-}Fair}}$.